

第6章 分支限界法



学习要点

- 理解分支限界法的剪枝搜索策略
- 掌握分支限界法的算法框架
 - (1) 队列式(FIFO)分支限界法
 - (2) 优先队列式分支限界法

例子:

- (1) 0-1背包问题
- (2) 单源最短路径问题
- (3) 旅行售货员问题
- (4) 装载问题



基本思想

类似于回溯法，分支限界法也是一种在问题的解空间树上搜索问题解的算法。

分支限界法与回溯法对比

	回溯法	分支限界法
目标/应用范围	能够找出满足约束条件的所有解	找出满足约束条件的一个解或特定意义下的最优解
解空间树的搜索方式	深度优先	广度优先或最小耗费优先
存储结点的数据结构	搜索过程中动态产生问题的解空间	队列、优先队列
结点存储特性	只保存根结点到当前扩展结点的路径	每个结点只有一次成为扩展结点（活结点）的机会

广度优先搜索 (BFS)

广度优先搜索 (Breadth First Search, BFS) 类似于树的按层次遍历的过程，是另外一种通用的图和树的遍历方法。给定图 $G = (V, E)$ ，BFS基本思想为：

□ 初始化

任选一个结点 v 作为源结点，BFS先访问**源结点** v ，并将其标记为**已访问**

□ 层次遍历

“由近及远，按层展开”

1. 从 v 出发，依次访问 v 的邻接点： $w_1, w_2, \dots, w_t$ ，如果 w_i ($i = 1, 2, \dots, t$) **未访问**过，则标记 w_i 为**已访问**，并**插入**到**先进先出**的队列；
2. 取出并**扩展**队列的**头结点**，把它**未访问的子结点****插入**队列；
3. 依此类推，直到图中所有和源结点 v 有路径相通的结点均已访问过为止。

□ 循环处理

若图 G 中仍然存在未访问的结点，则另选一个未访问的结点作为**新的源结点**重复上述过程，直到图中所有结点均被访问过为止。

- 扩展结点 (E-结点, Expansion Node)

- ✓ 一个正在产生儿子的结点

- 活结点 (L-结点, Live Node)

- ✓ 已生成但尚未被扩展的结点 (即尚未检查其子结点是否符合条件)

- 死结点 (D-结点, Dead Node)

- ✓ 一个所有儿子已经产生的结点, 也称为已扩展结点。

广度优先搜索 (BFS)

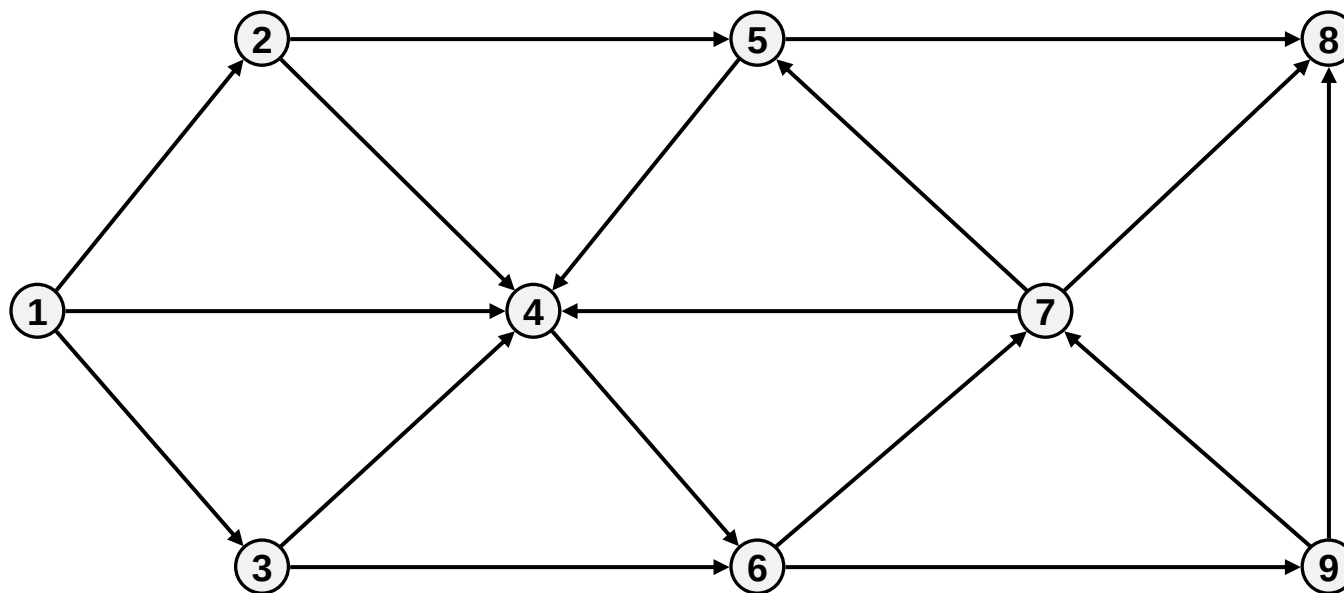
基于队列的BFS算法框架：

```
function BFS(problem, queue)
    node = Make-Node(Initial-State[problem]);
    queue ← Insert(node, queue);
    do while(1)
        if queue == Empty
            return failure
        node ← Remove-First(queue)
        visit(node);
        if State[node] == Goal
            return Solution(node)
        sonNodes = Expend(node, problem);
        queue ← Insert-All(sonNodes, queue)
```

//queue初始化为空
//生成初始结点
//队列中加入初始结点
//取出队列的头部结点
//访问当前结点
//目标测试
//搜索到目标状态，返回解
//生成node的未访问子结点
//依次插入所有子状态

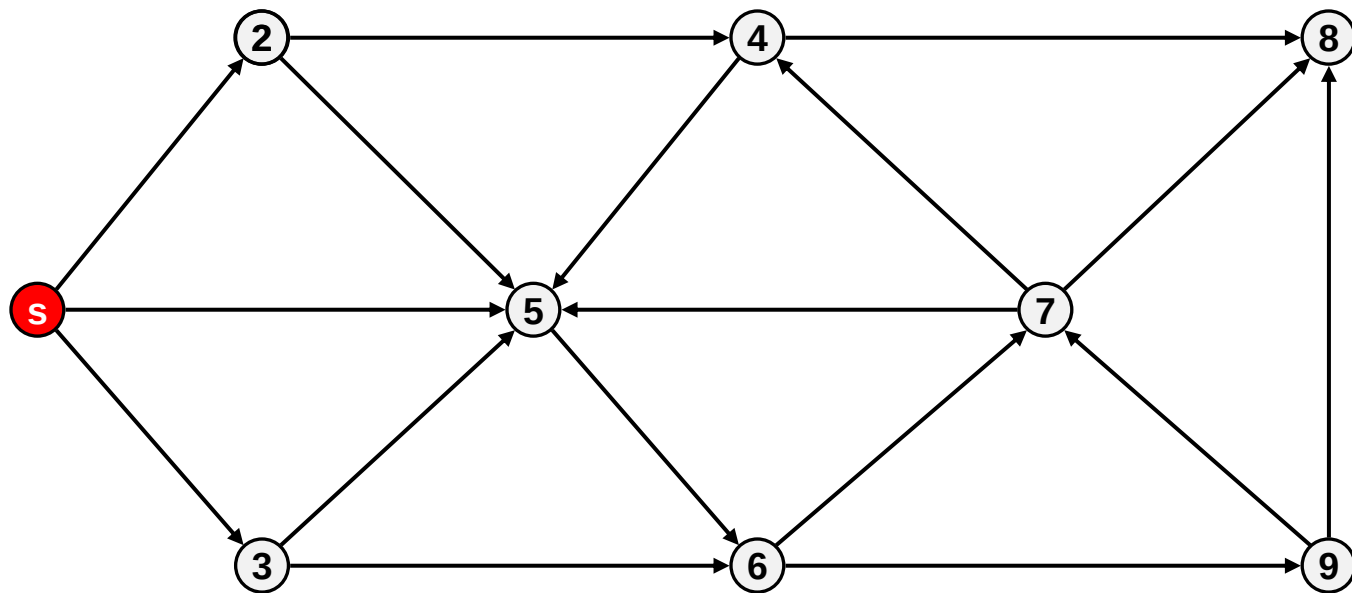
BFS 演示

【例】给定一个有向图 $G = (V, E)$ ，列出该图广度优先搜索的一个访问序列。



BFS 演示

初始化:



未访问结点

已访问结点

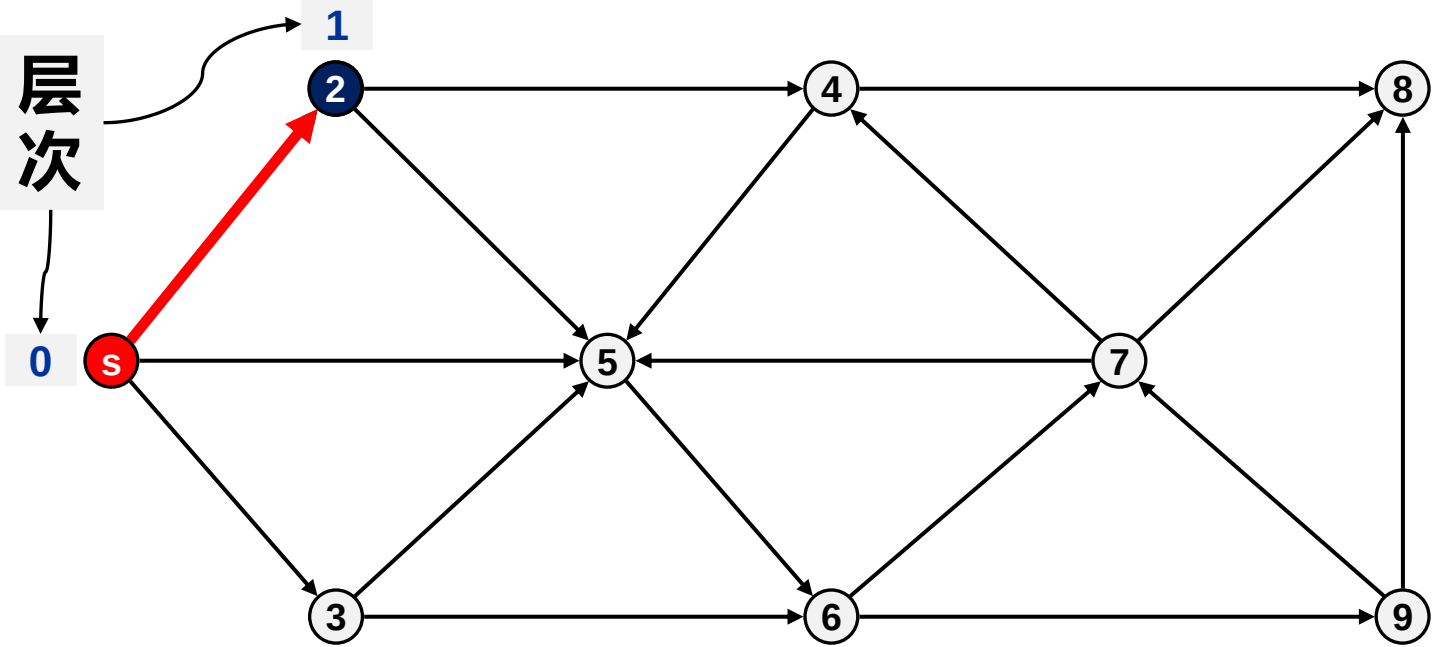
扩展结点

已扩展节点

Queue: s

BFS 演示

层次遍历:

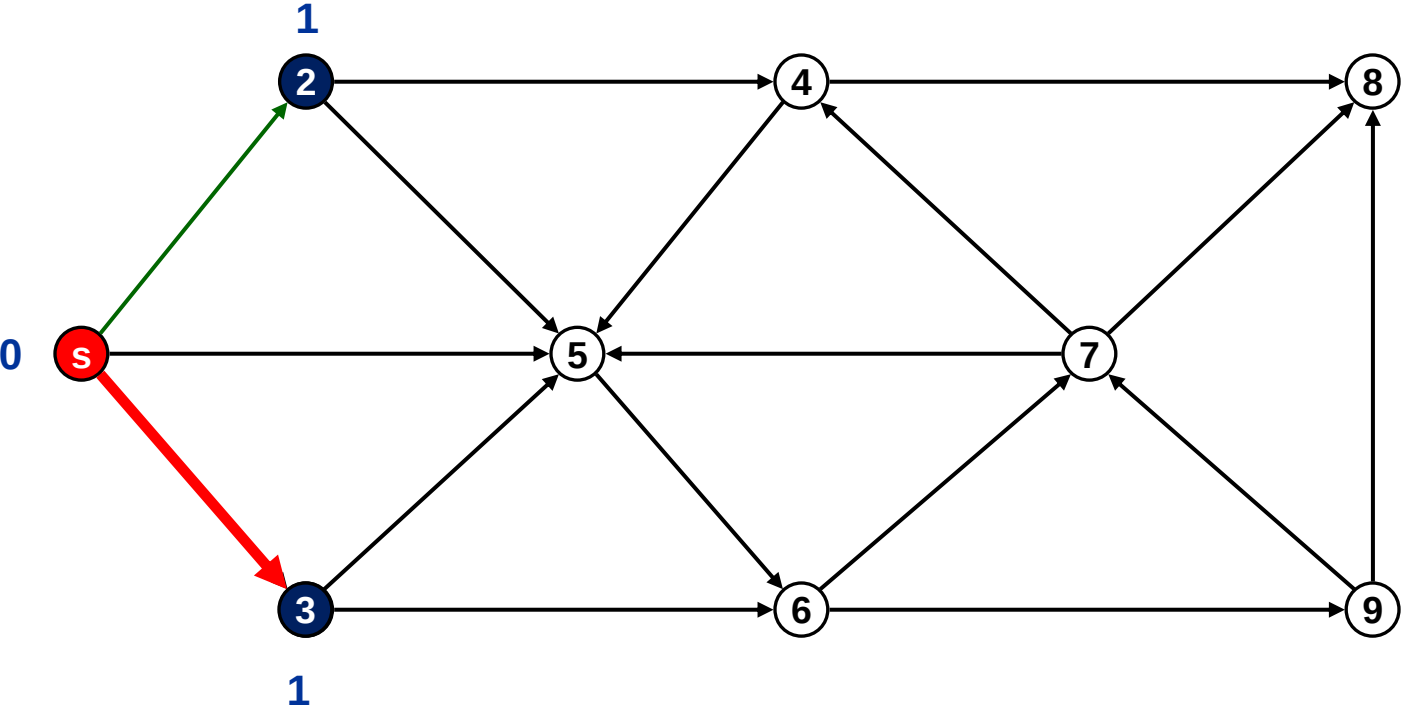


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: s 2

BFS 演示

层次遍历:

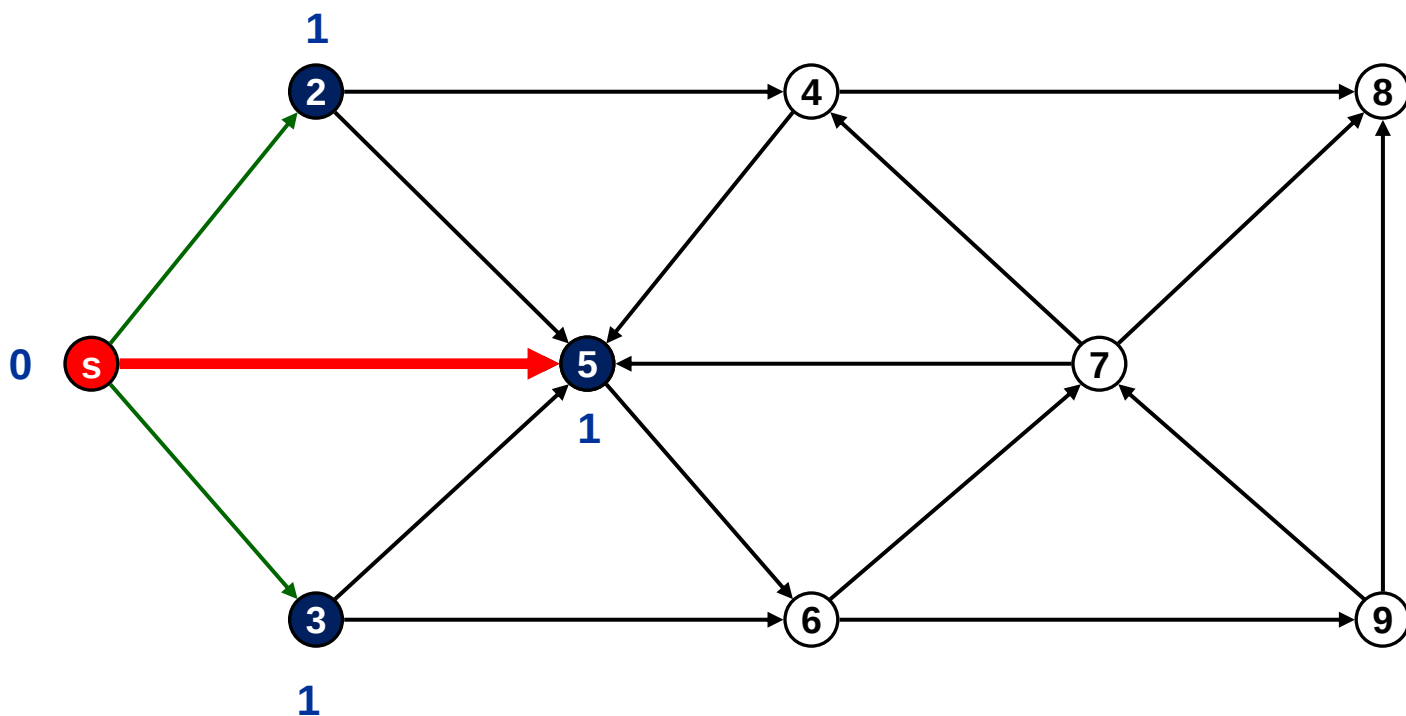


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: s 2 3

BFS 演示

层次遍历:



未访问结点

已访问结点

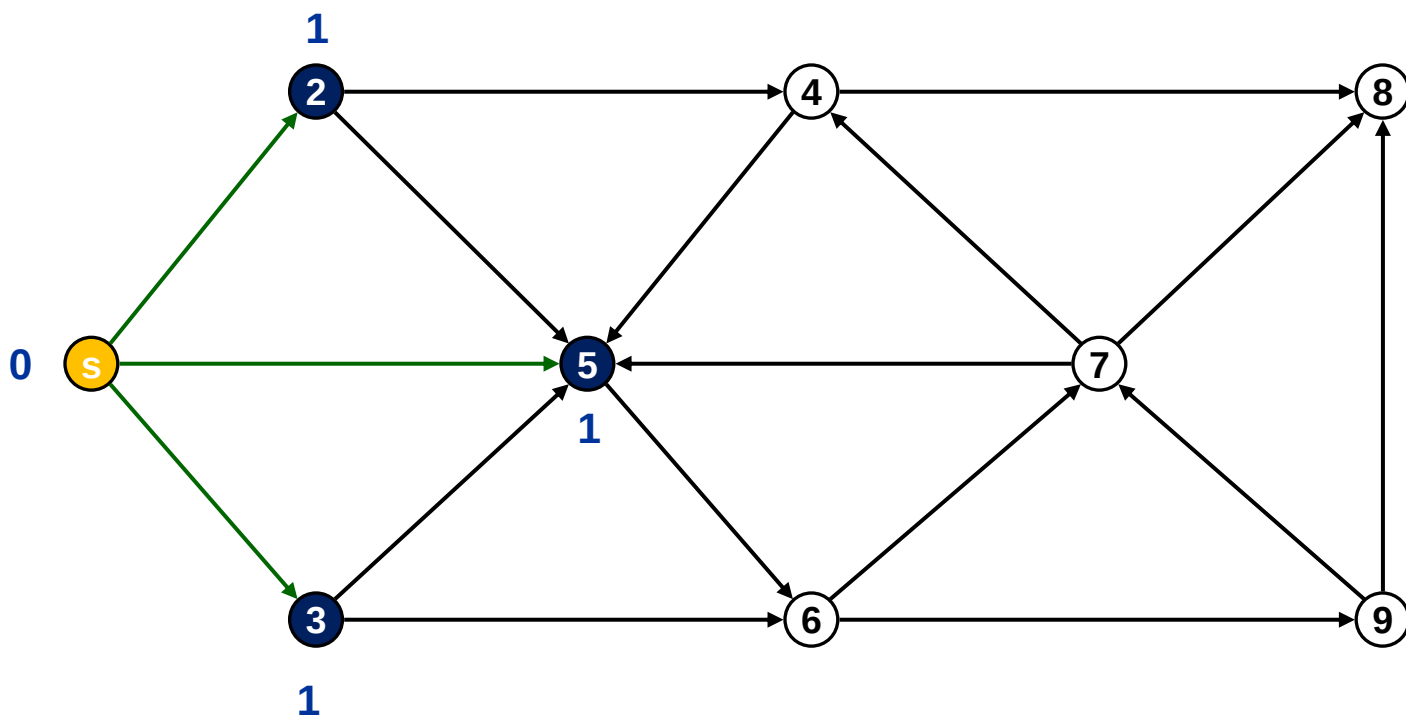
扩展结点

已扩展节点

Queue: s 2 3 5

BFS 演示

层次遍历:



未访问结点

已访问结点

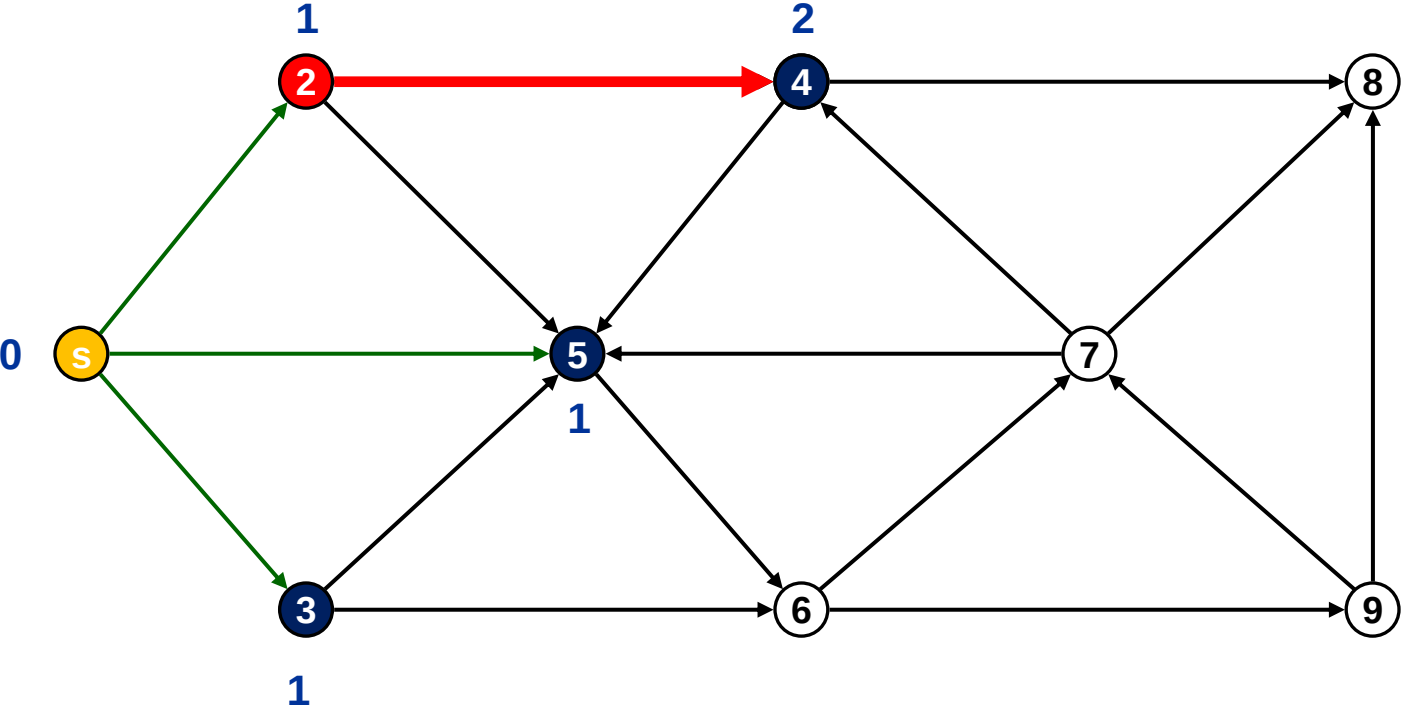
扩展结点

已扩展节点

Queue: s 2 3 5

BFS 演示

层次遍历:

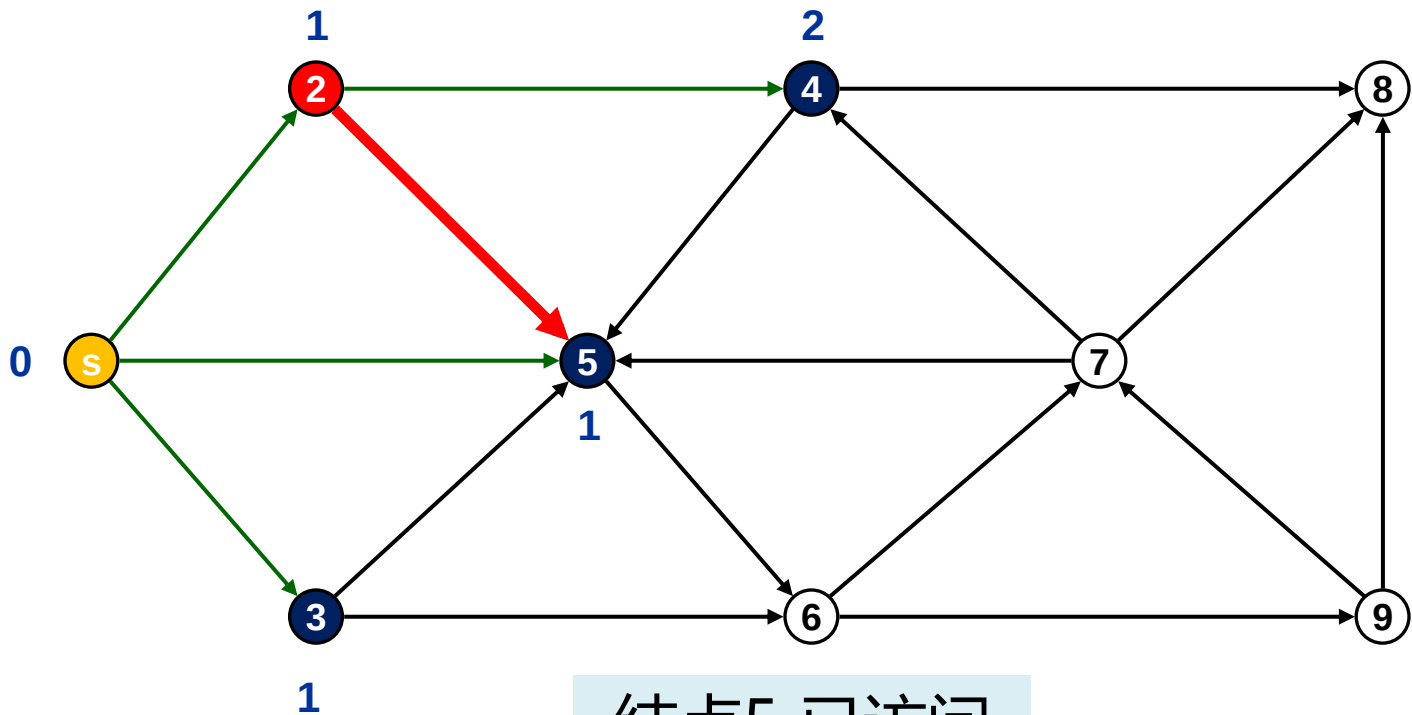


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 2 3 5 4

BFS 演示

层次遍历:



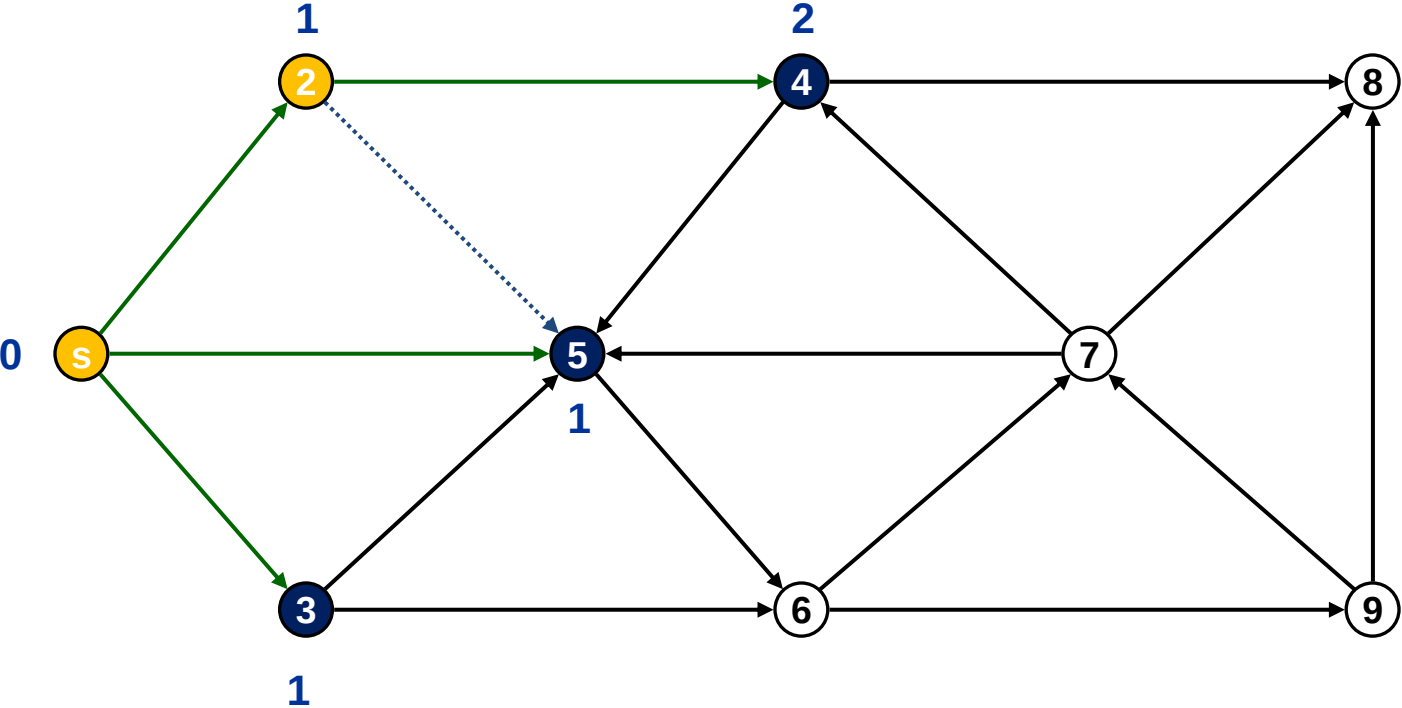
结点5 已访问
不再入队列

未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 2 3 5 4

BFS 演示

层次遍历:

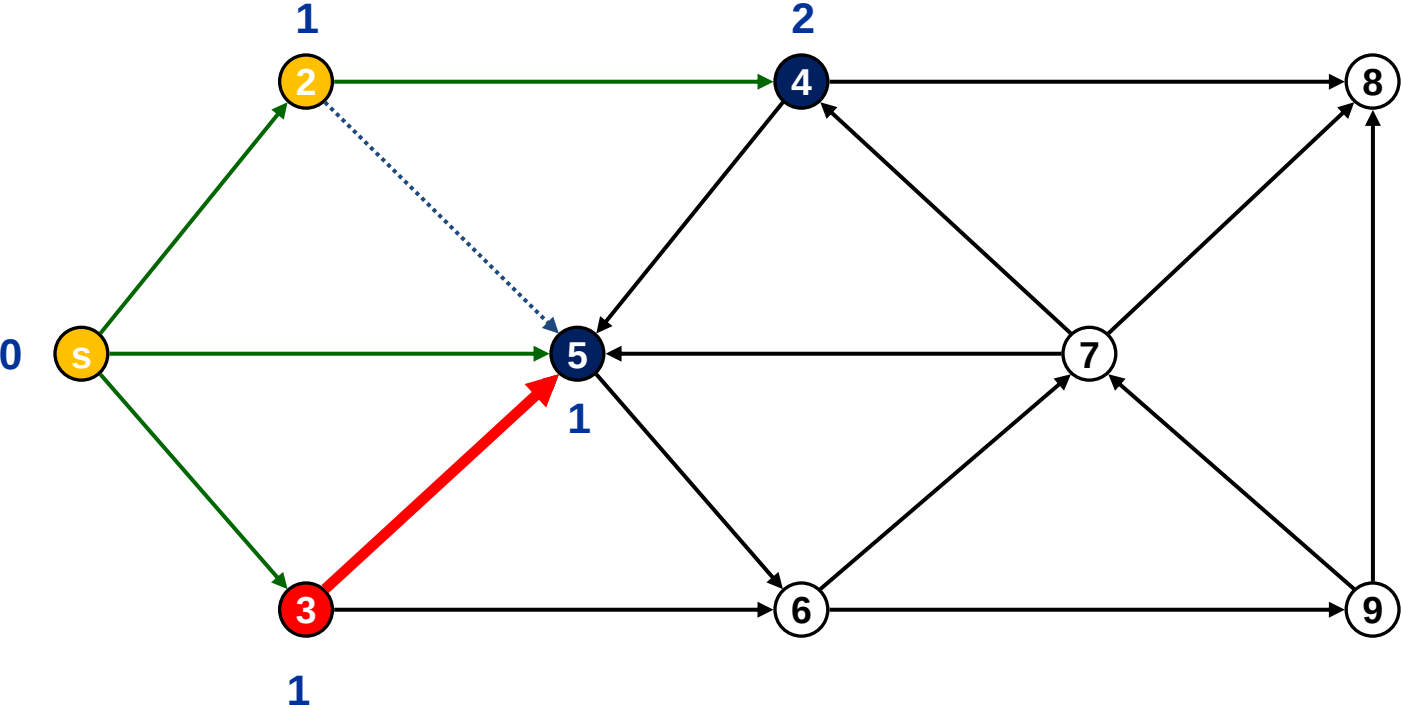


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 2 3 5 4

BFS 演示

层次遍历:

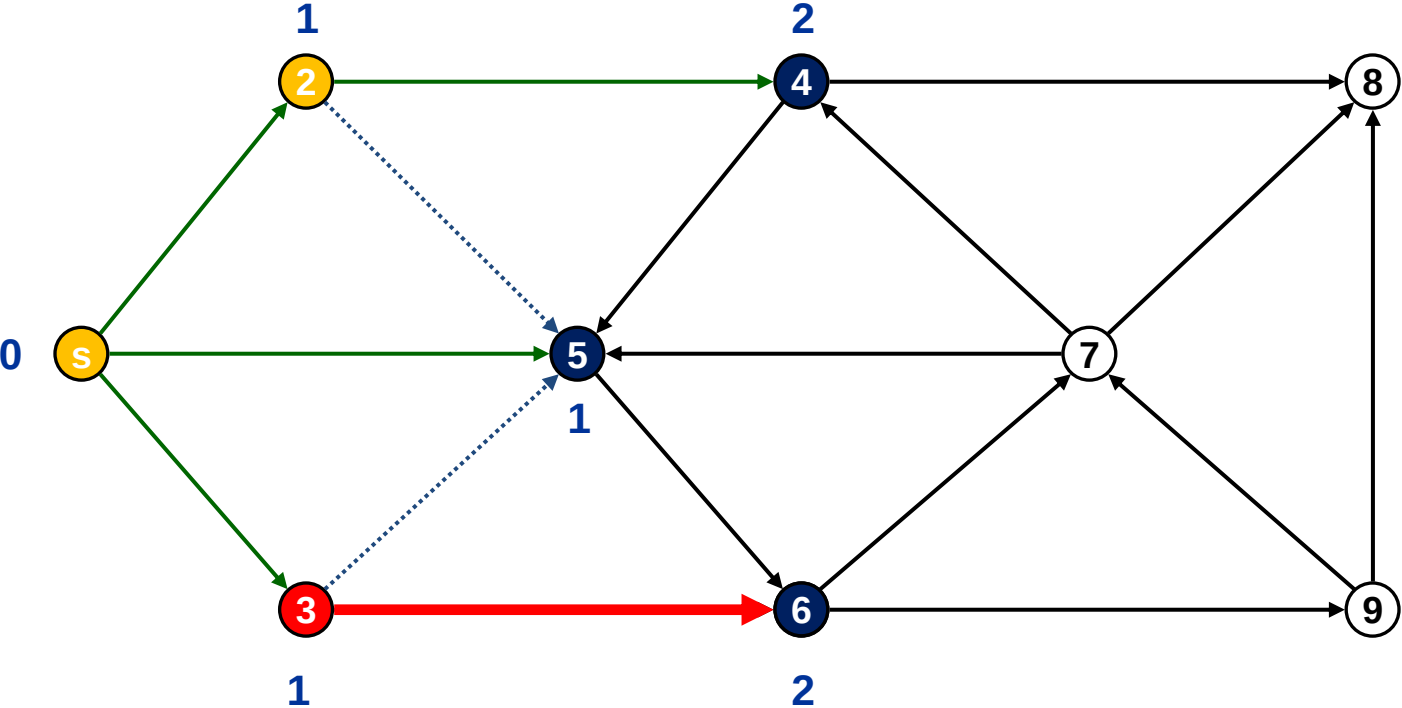


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 3 5 4

BFS 演示

层次遍历:

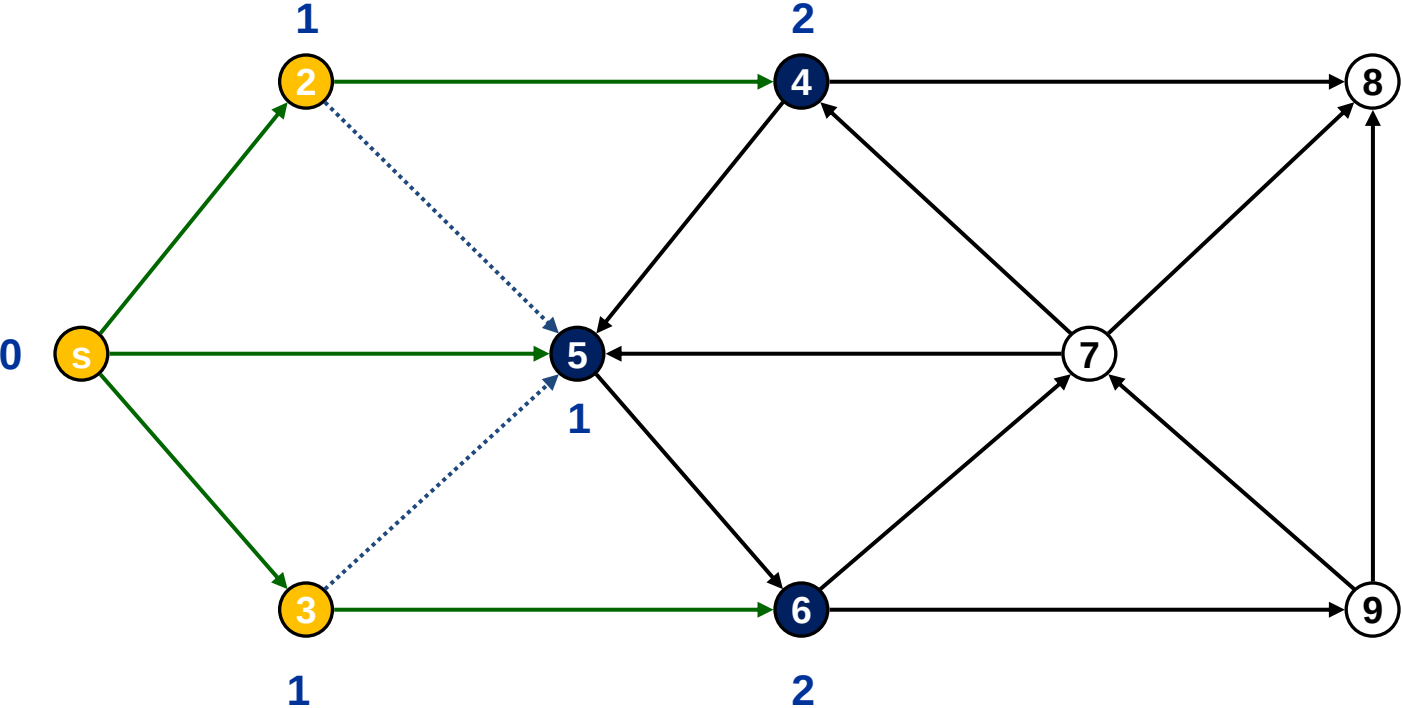


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 3 5 4 6

BFS 演示

层次遍历:

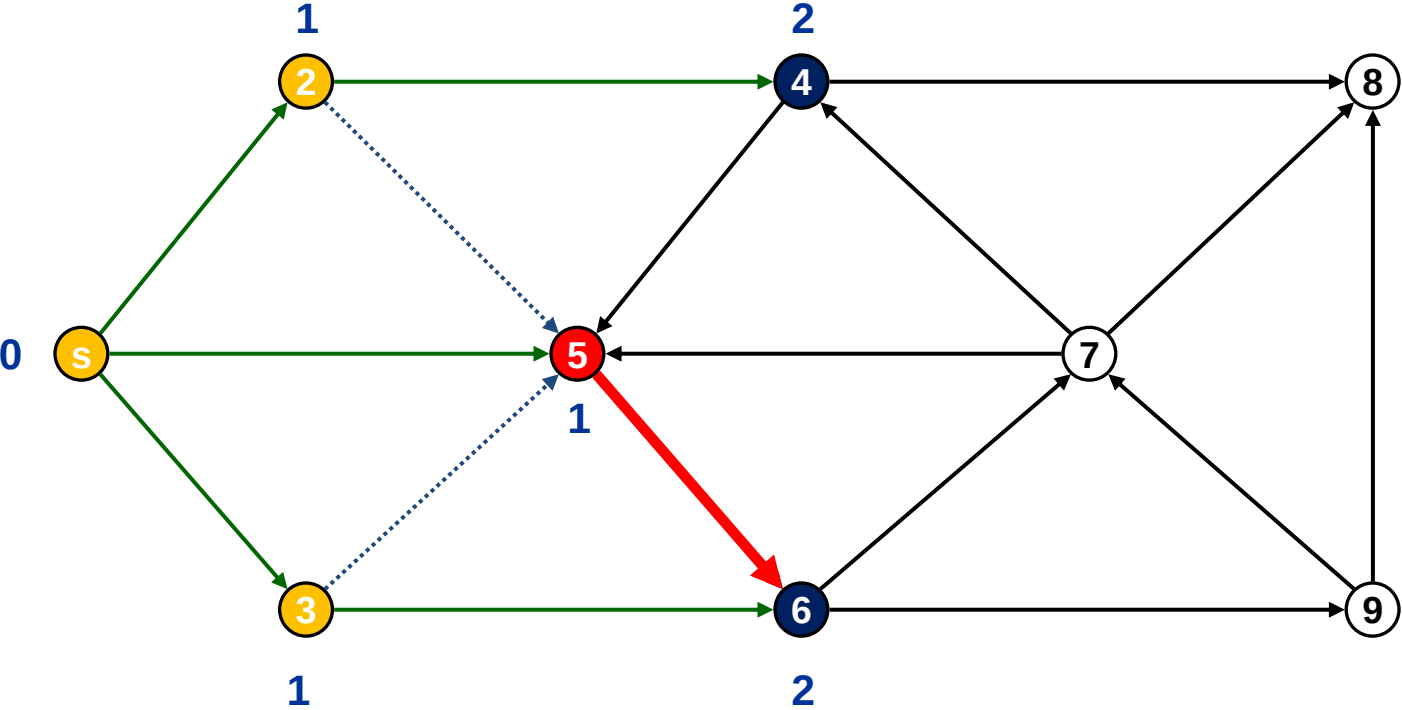


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 3 5 4 6

BFS 演示

层次遍历:

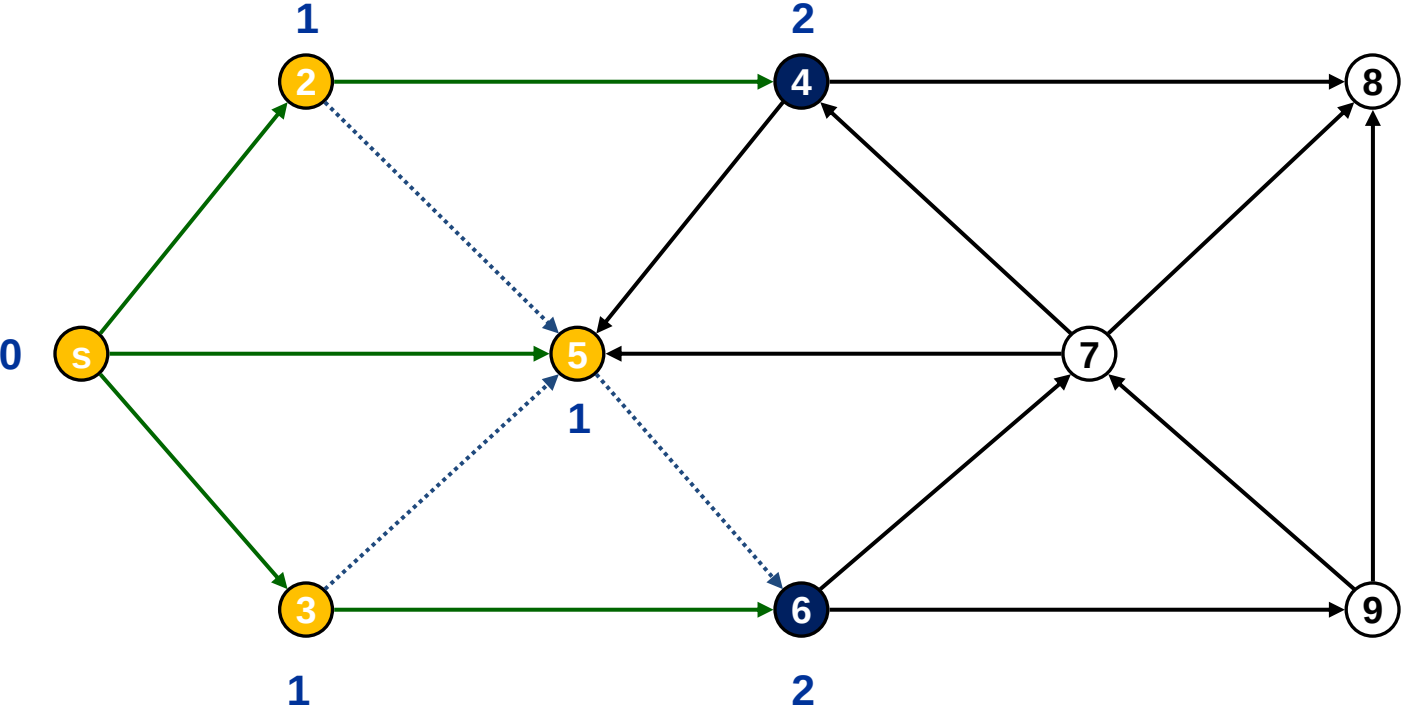


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 5 4 6

BFS 演示

层次遍历:

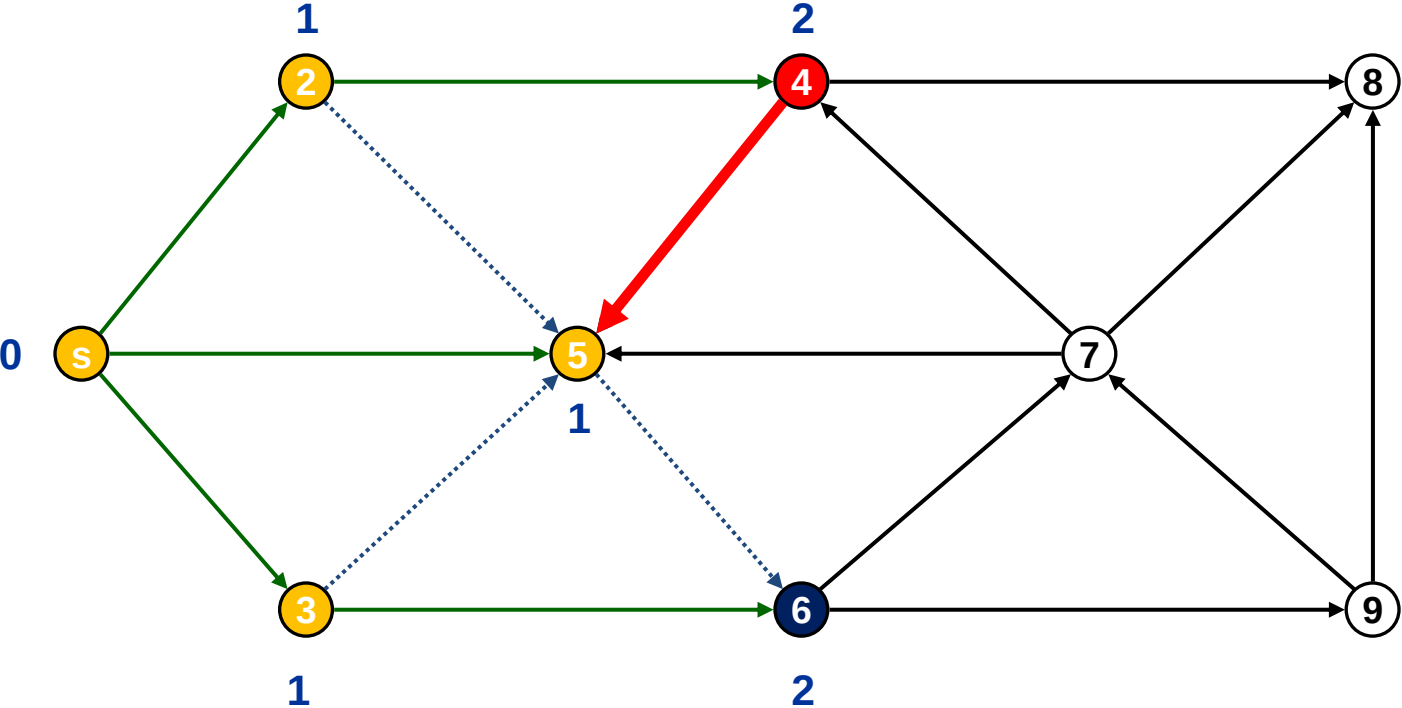


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 5 4 6

BFS 演示

层次遍历:

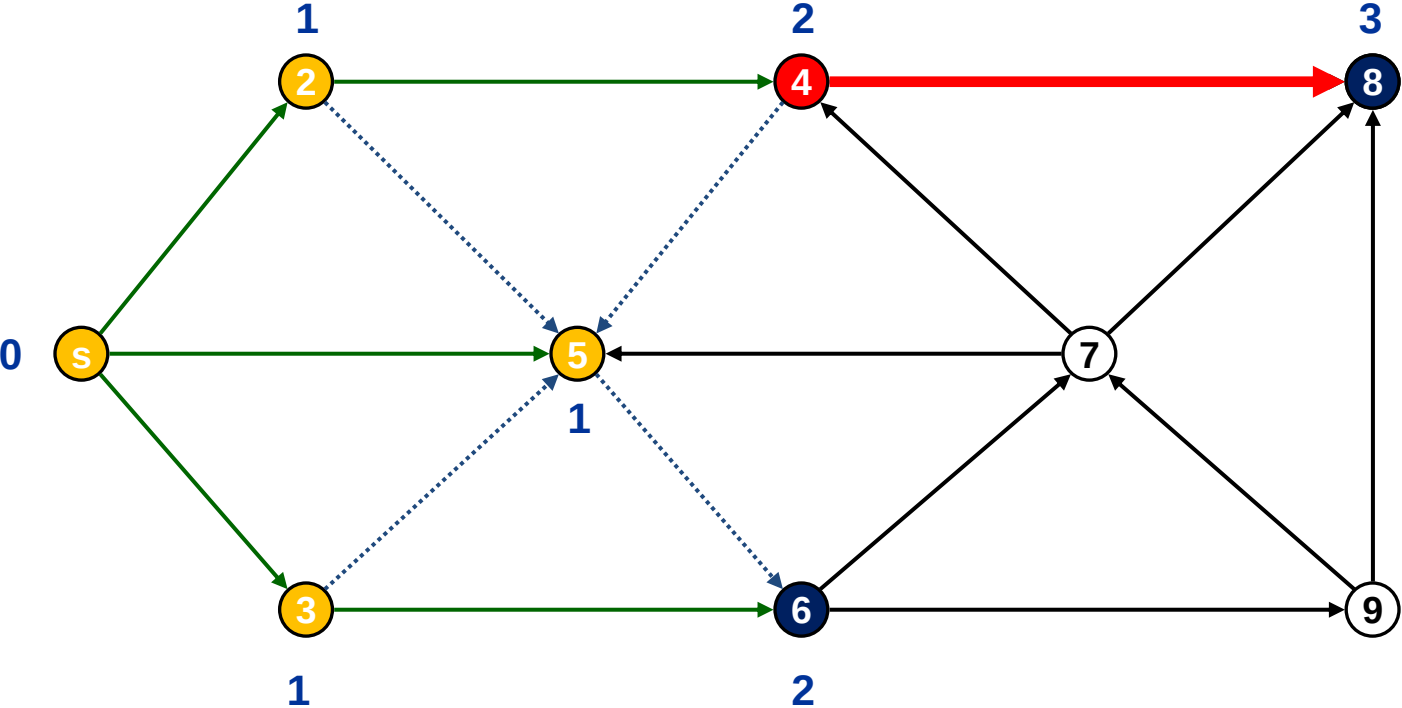


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 4 6

BFS 演示

层次遍历:

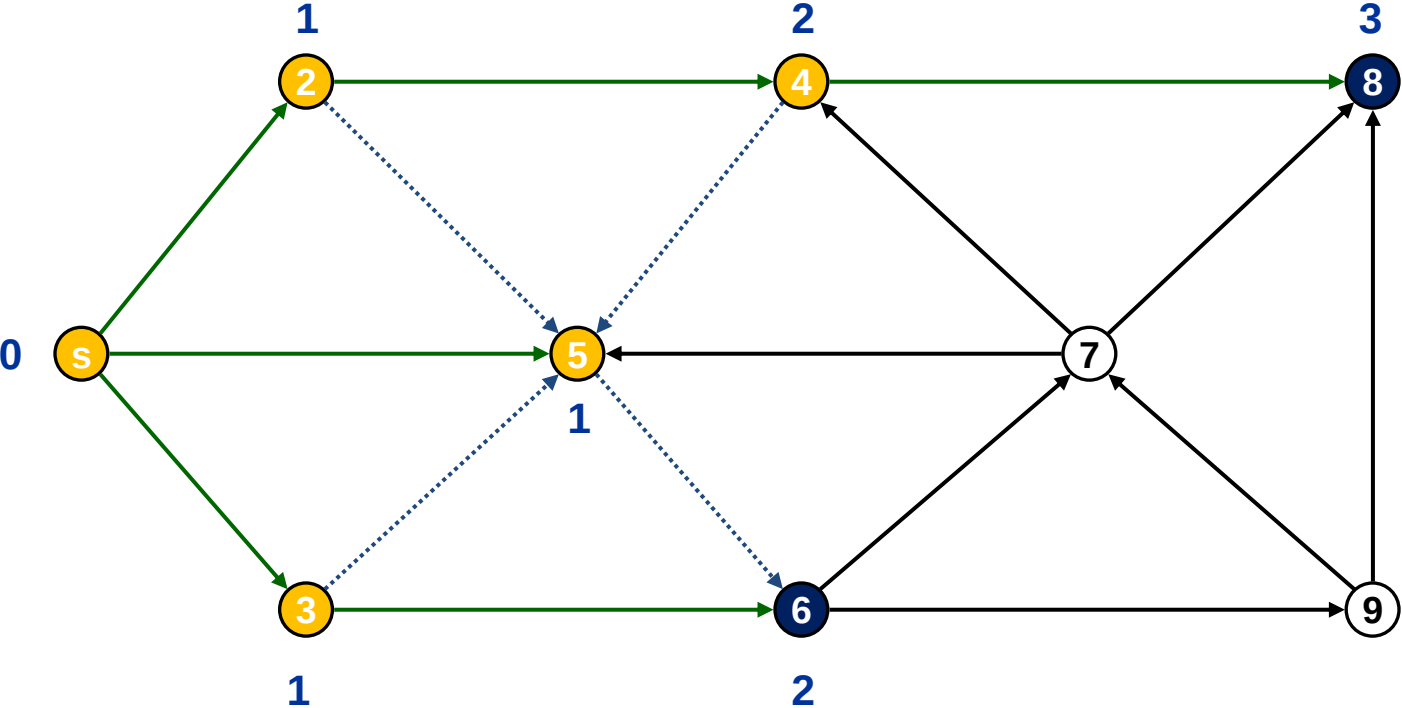


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 4 6 8

BFS 演示

层次遍历:

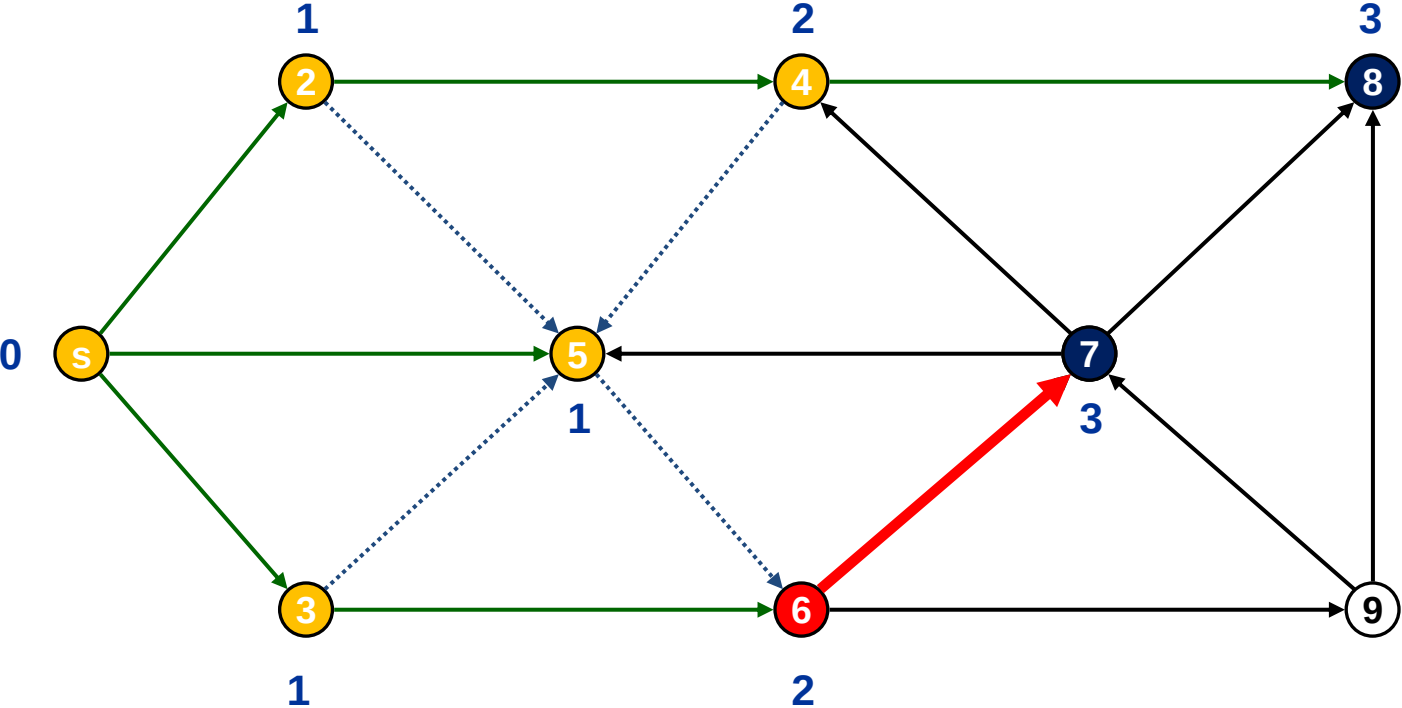


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 4 6 8

BFS 演示

层次遍历:

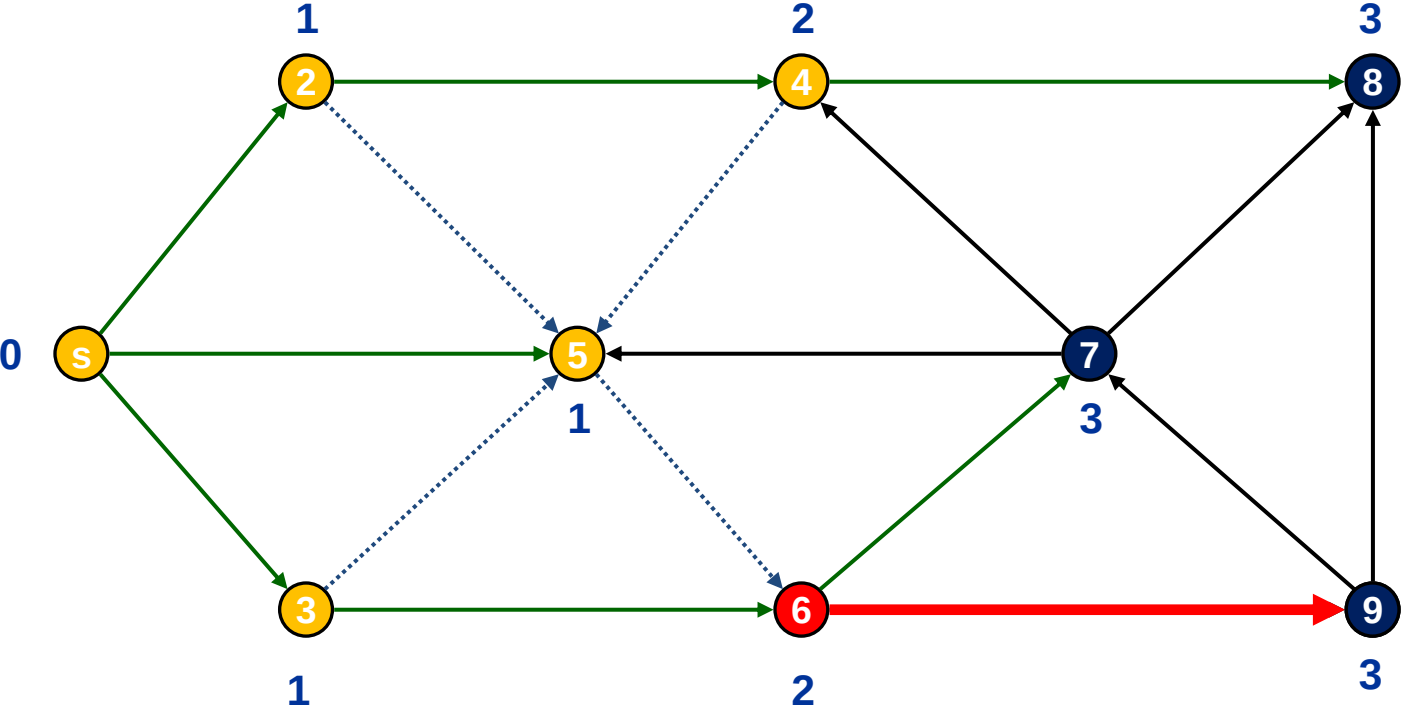


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 6 8 7

BFS 演示

层次遍历:

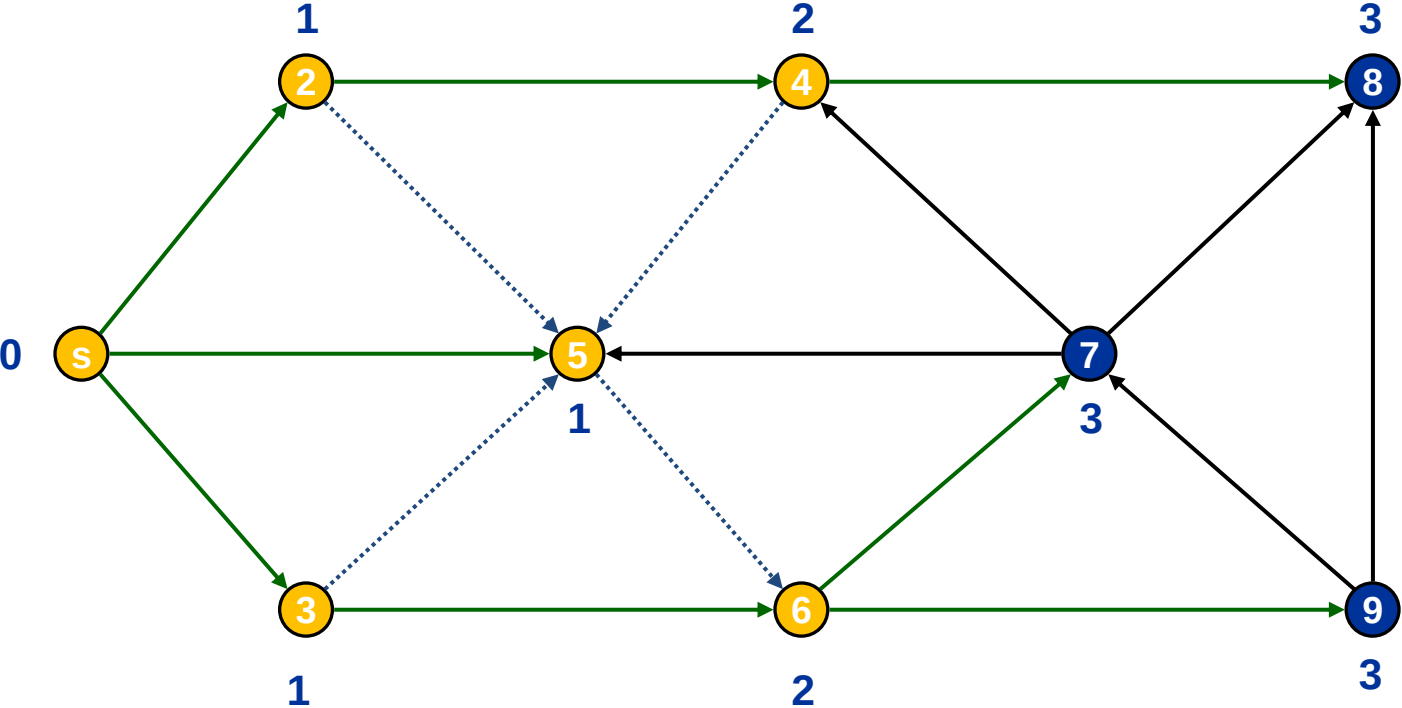


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 6 8 7 9

BFS 演示

层次遍历:

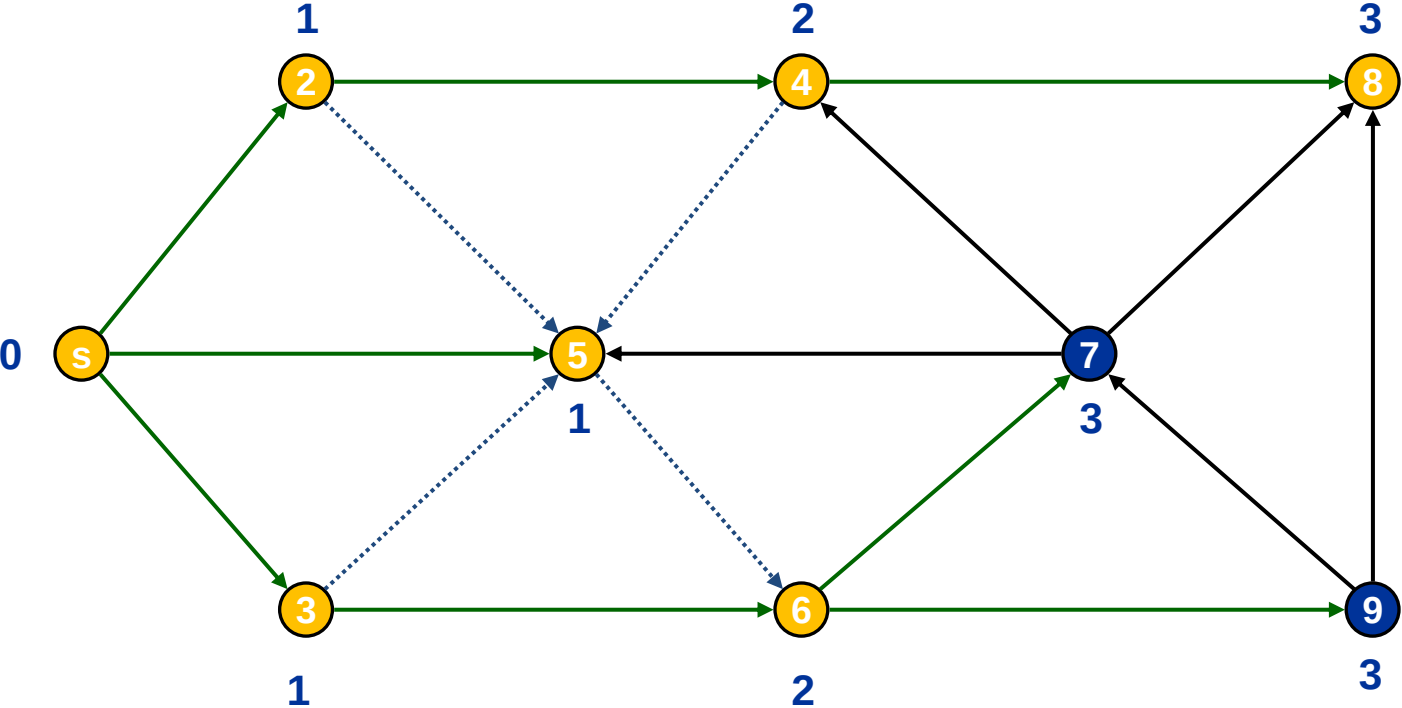


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 6 8 7 9

BFS 演示

层次遍历:

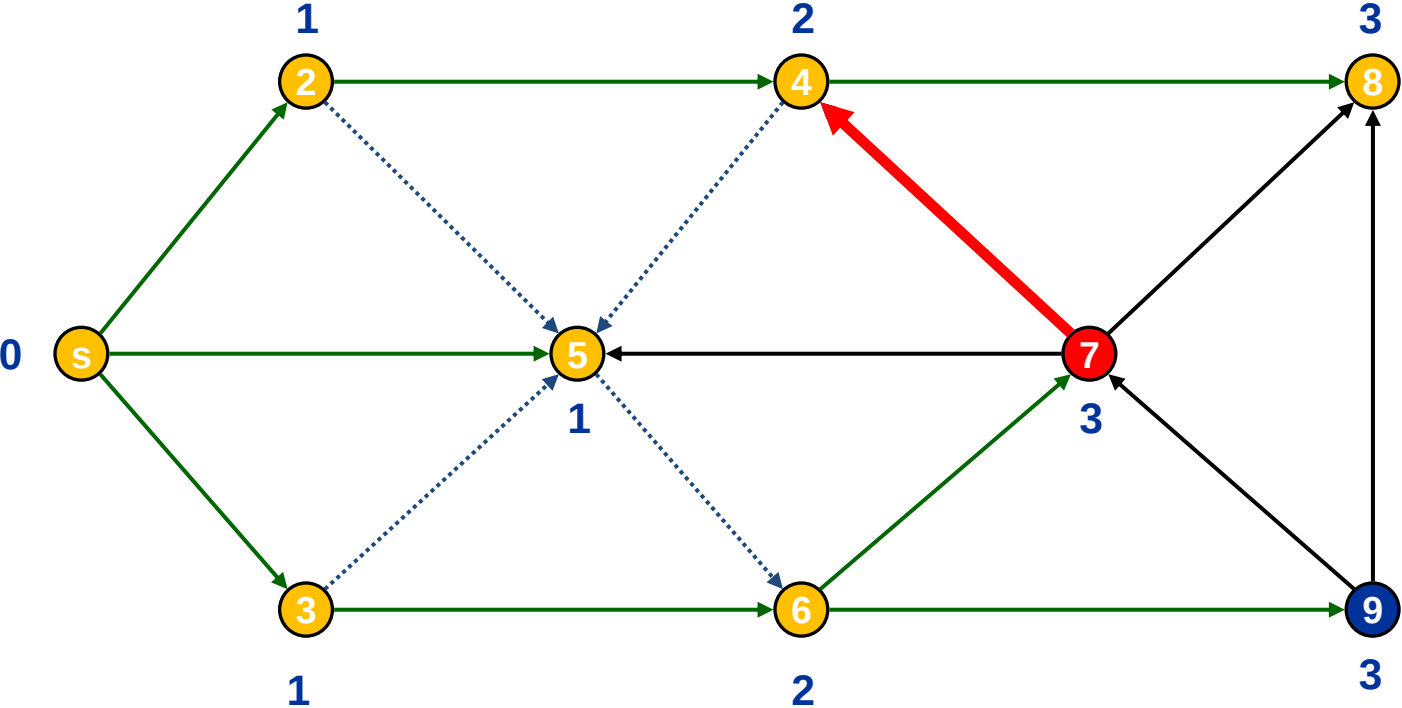


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 8 7 9

BFS 演示

层次遍历:

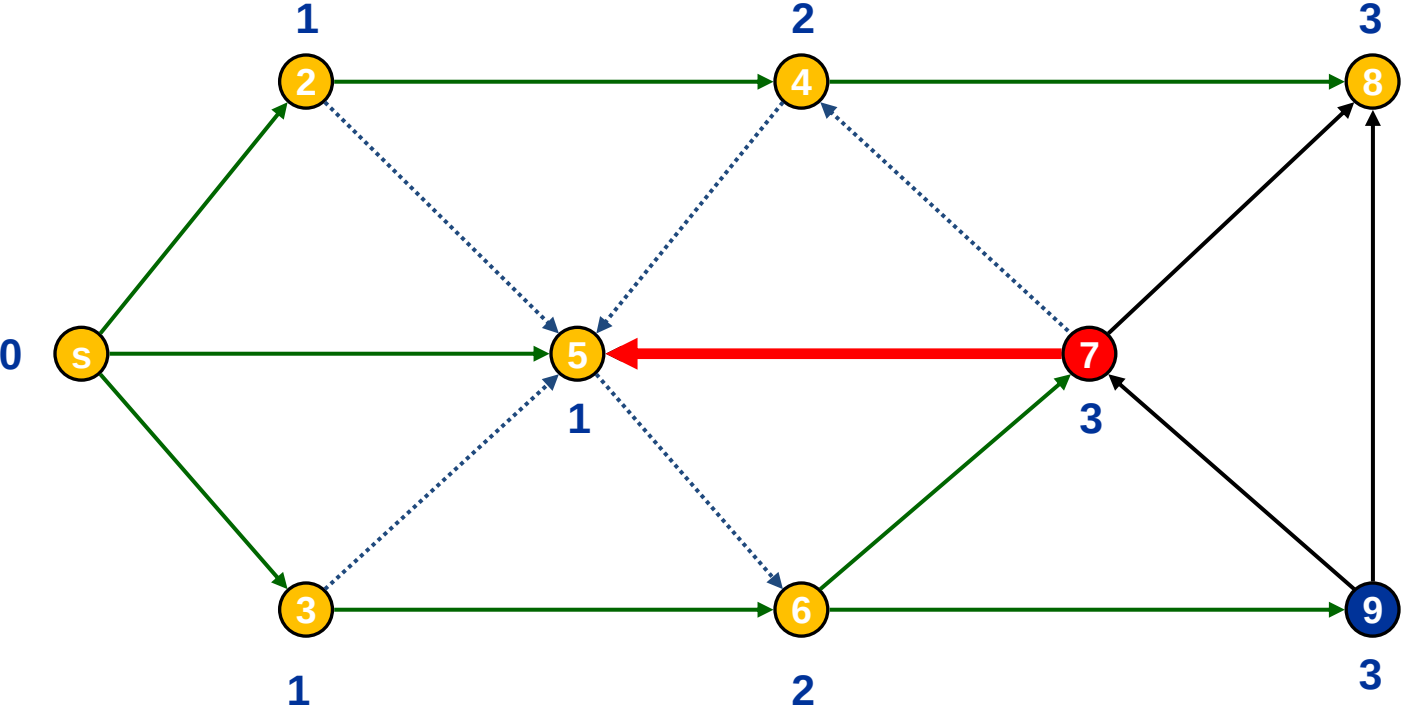


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 7 9

BFS 演示

层次遍历:

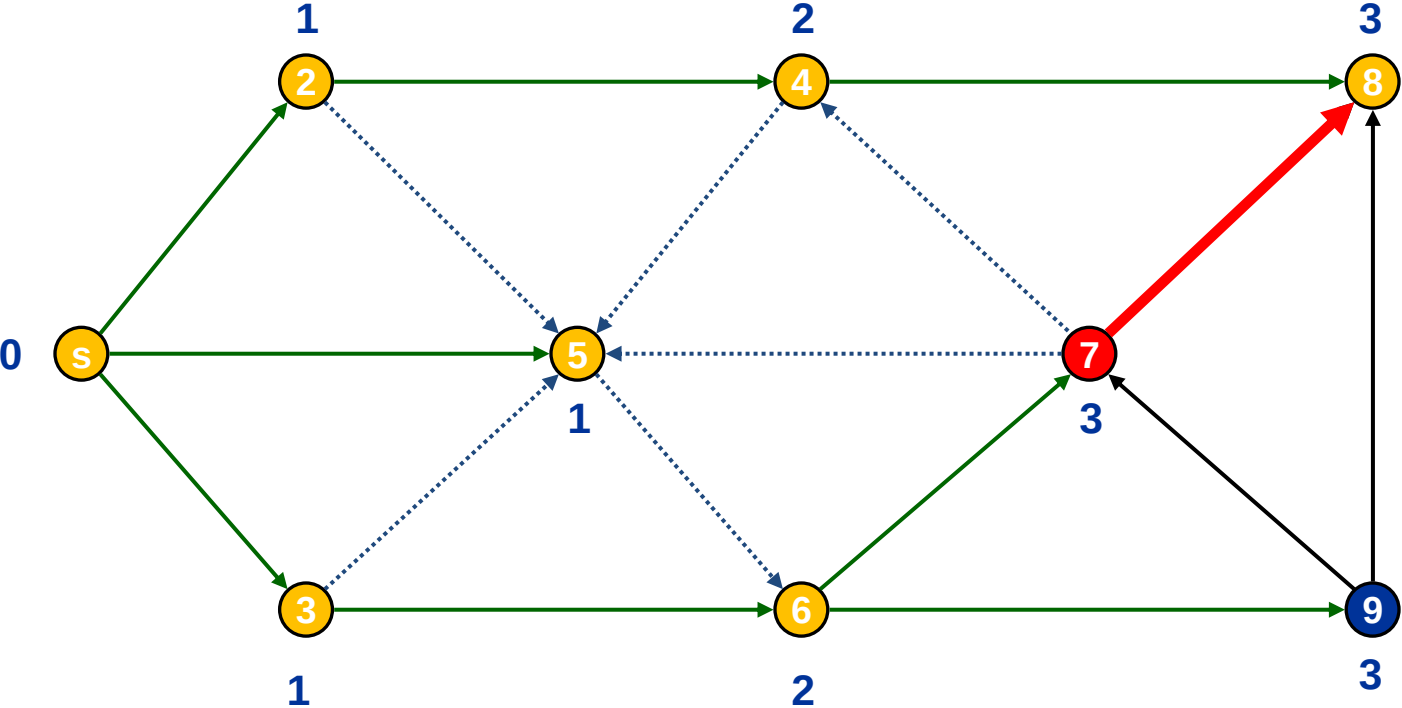


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 7 9

BFS 演示

层次遍历:

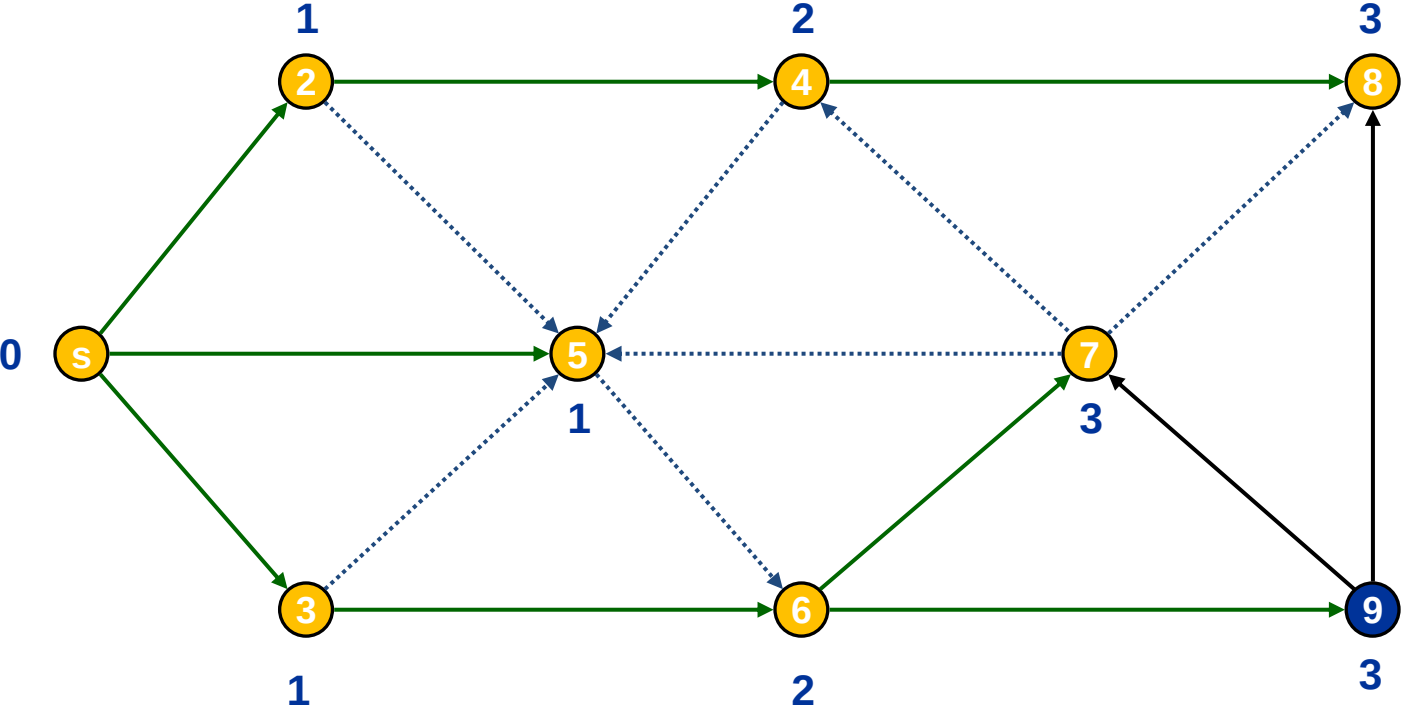


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 7 9

BFS 演示

层次遍历:

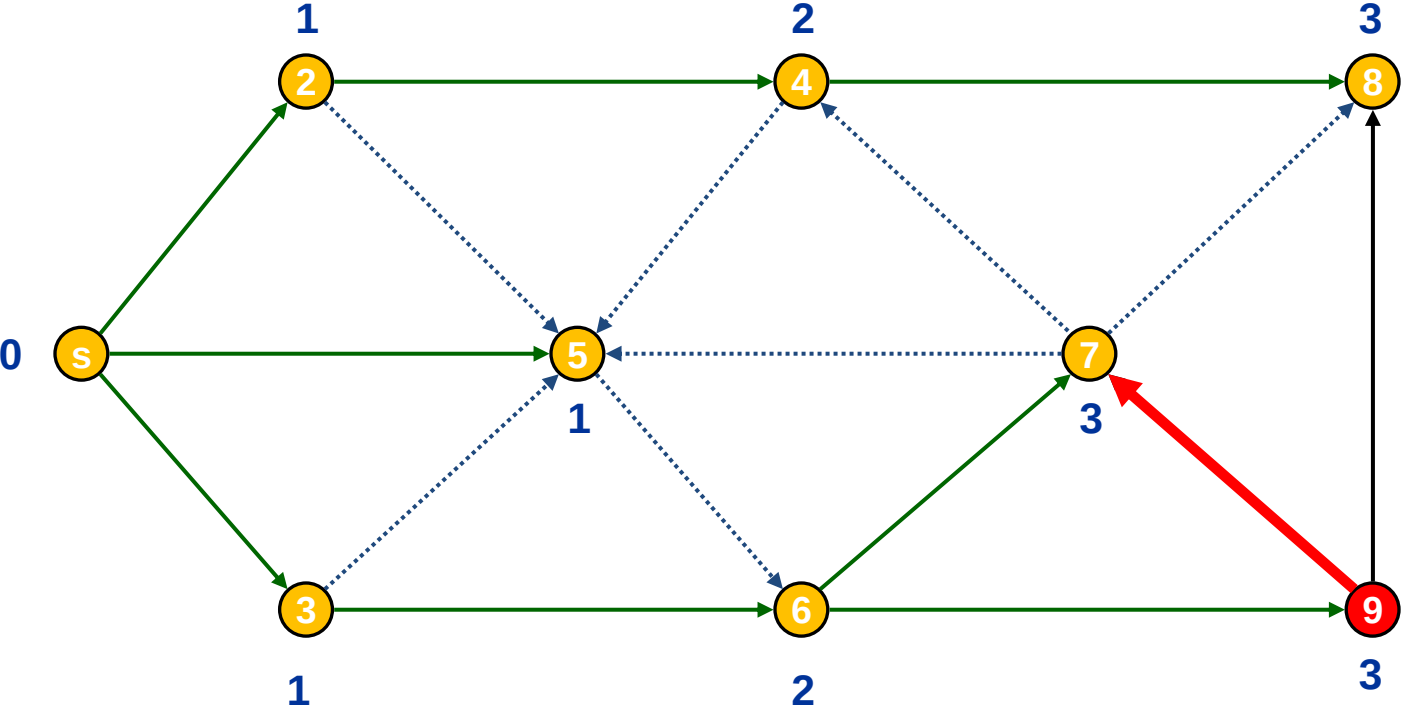


未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 7 9

BFS 演示

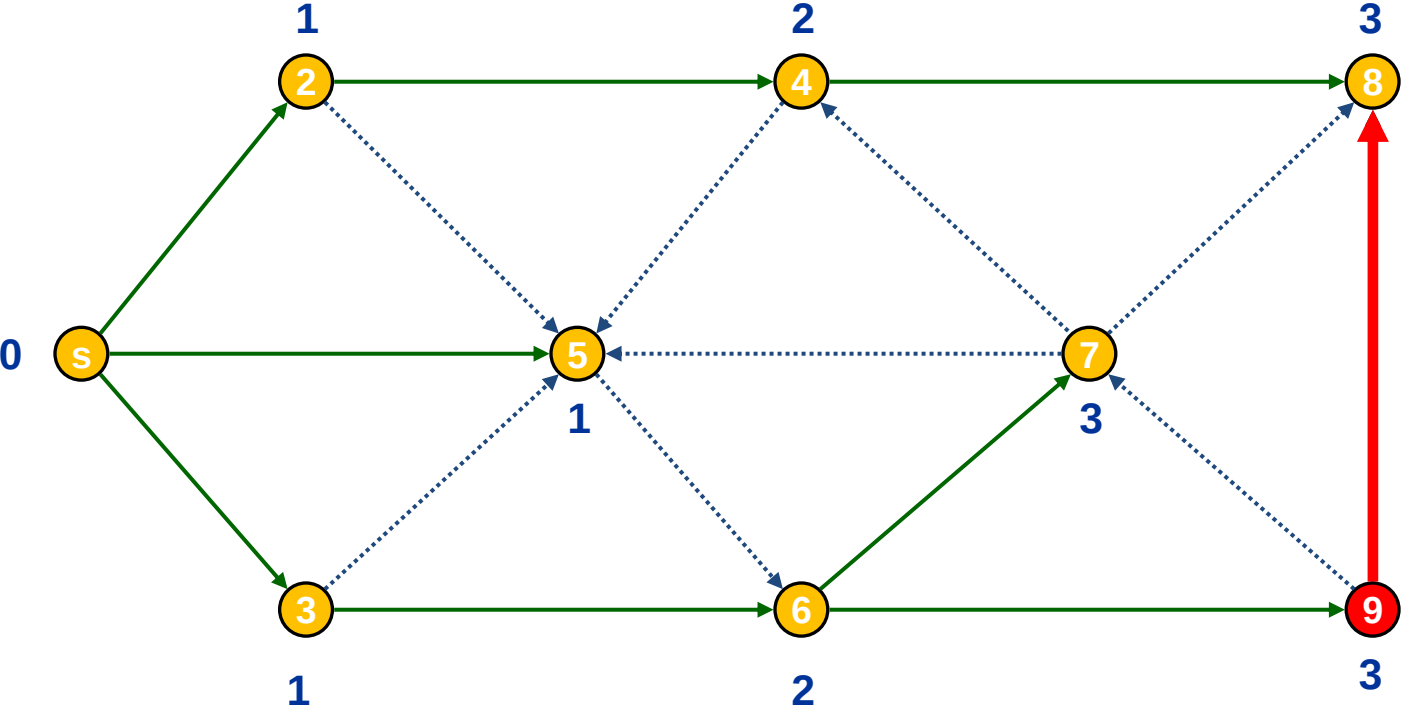
层次遍历:



未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 9

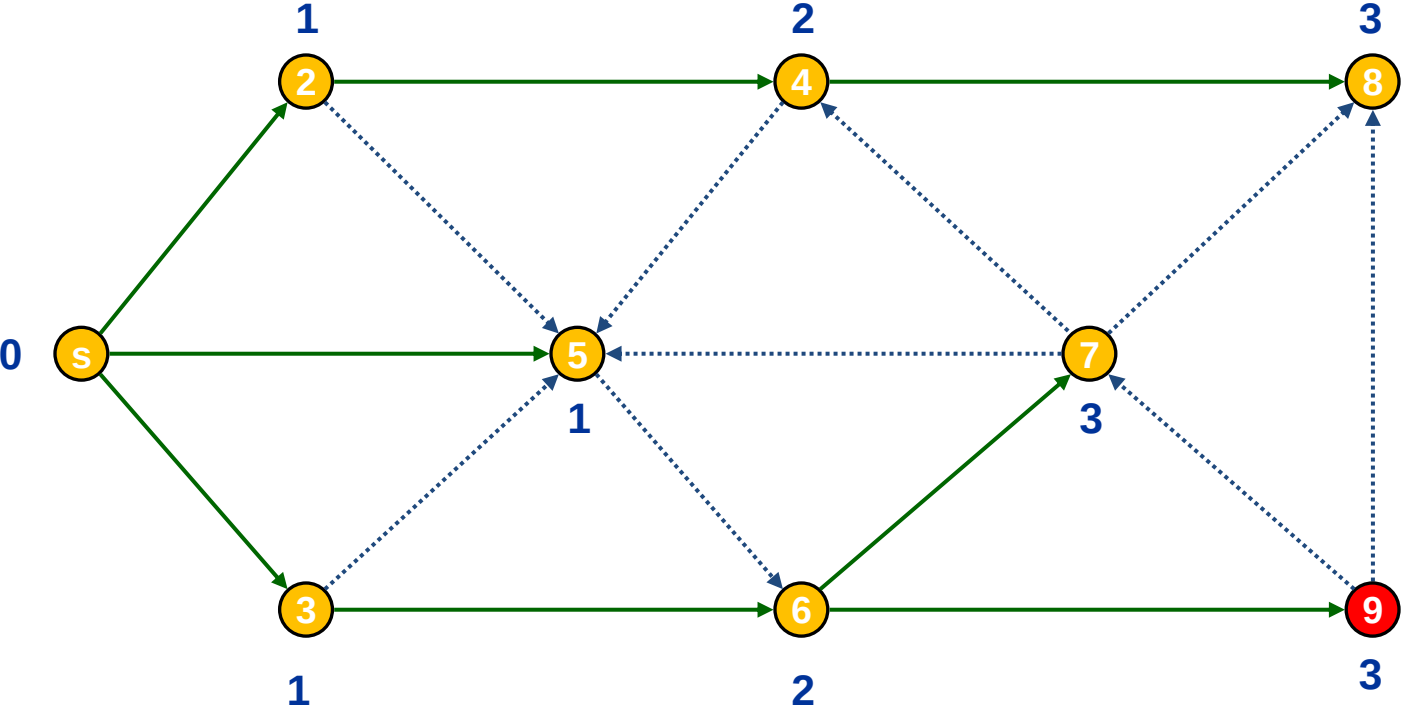
BFS 演示



未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 9

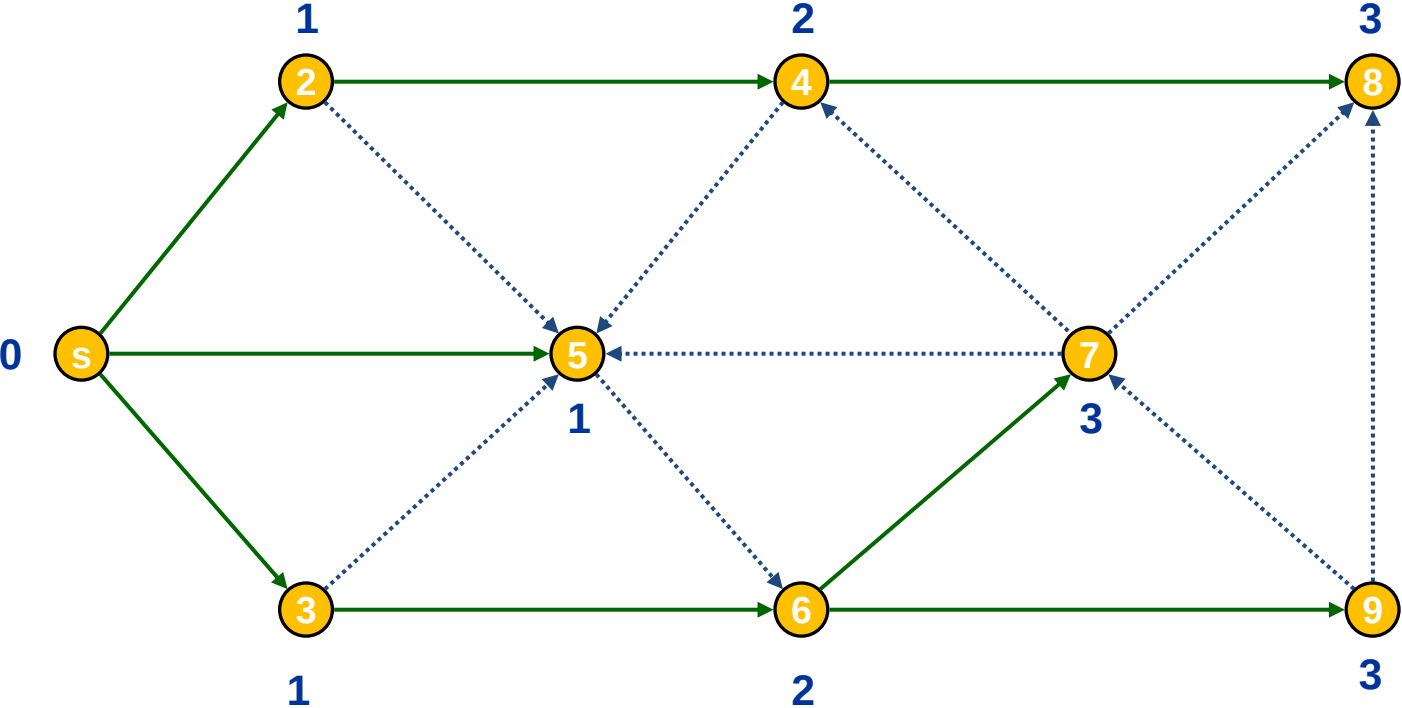
BFS 演示



未访问结点
已访问结点
扩展结点
已扩展节点

Queue: 9

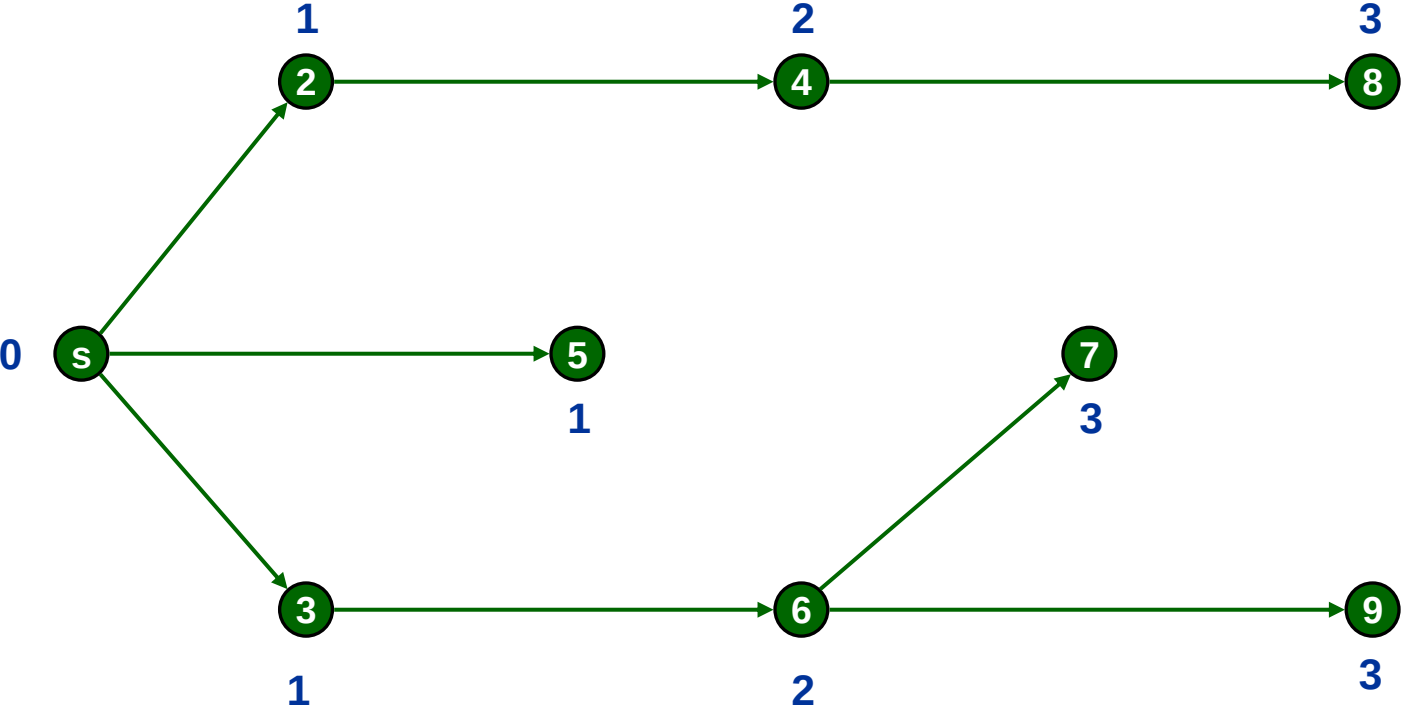
BFS 演示



未访问结点
已访问结点
扩展结点
已扩展节点

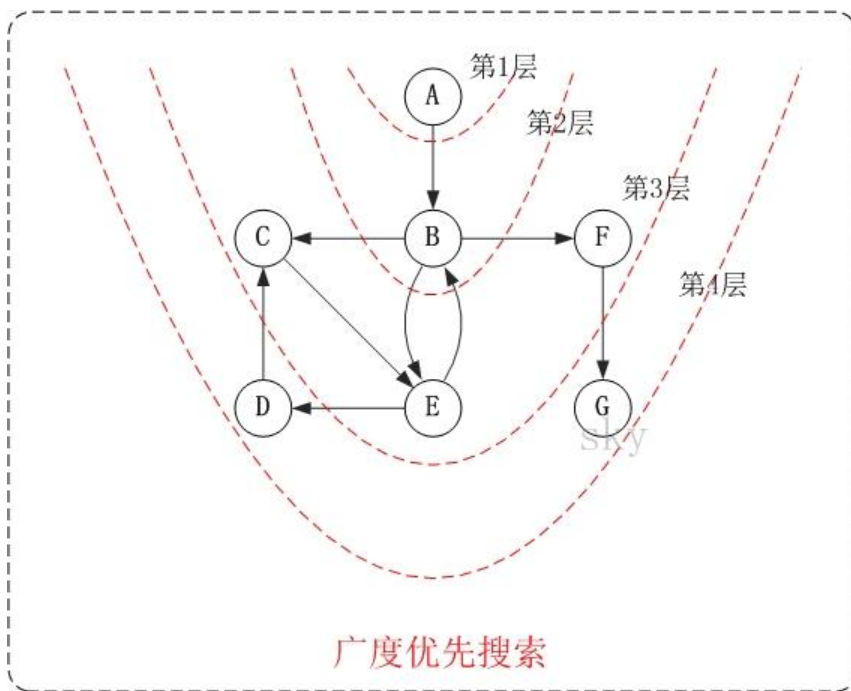
Queue:

BFS 演示



BFS搜索层次图

广度优先搜索



Step 1: 访问A。

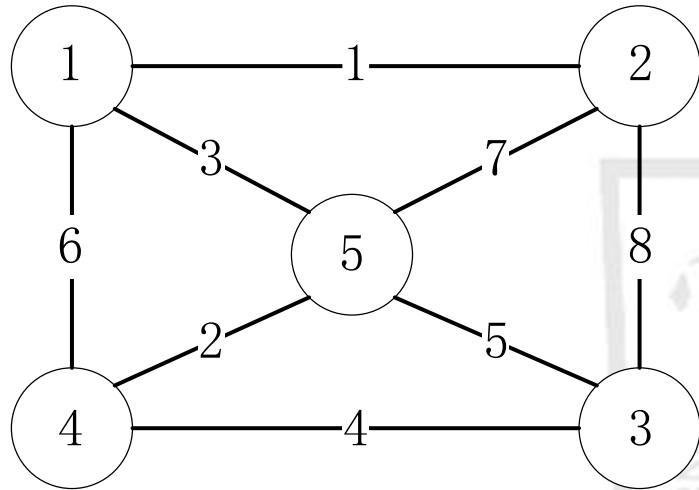
Step 2: 访问B。在访问了B之后，接下来访问B的出边的另一个顶点，即C，E，F。由于顶点ABCDEFG按照顺序存储，因此会先访问C，再依次访问E，F。

Step 3: 依次访问C，E，F。在访问完C，E，F之后，再依次访问它们的出边的另一个顶点。还是按照C，E，F的顺序访问，C的已经全部访问过了，那么就只剩下E，F；先访问E的邻接点D，再访问F的邻接点G。

Step 4: 依次访问D，G。

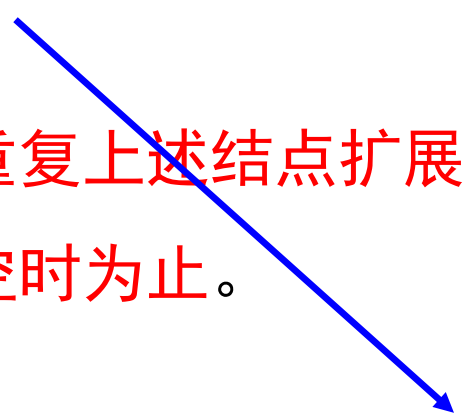
访问顺序：A -> B -> C -> E -> F -> D -> G

小试牛刀



给出所有从顶点2出发的广度
优先遍历序列？

- 
- 分支限界法以广度优先或最小耗费（最大效益）优先方式搜索问题的解空间树。
 - 每一个活结点只有一次机会成为扩展结点。
 - 活结点一旦成为扩展结点，就一次性产生其所有儿子结点。
 - 儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。
 - 从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。
 - 整个过程一直持续到找到所需的解或活结点表为空时为止。



剪枝

定义

分支限界法：采用广度优先产生状态空间树的结点，并使用剪枝函数的方法。

- “分支”：采用广度优先策略，依次生成扩展结点的所有分支（即，儿子结点）。
- “限界”：在结点的扩展过程中，计算结点的上界（或下界），边搜索边剪枝，以提高搜索效率。

0-1 背包问题

假设有4个物品，其重量分别为(4, 7, 5, 3)，价值分别为(40, 42, 25, 12)，背包容量 $C = 10$ 。

回顾：动态规划法、回溯法

分支限界？

分支 限界

广度优先 剪枝函数





假设有4个物品，其重量分别为(4, 7, 5, 3)，价值分别为(40, 42, 25, 12)，背包容量 $C = 10$ 。

将给定物品按单位重量价值从大到小排序，结果如下：

物品	重量(w)	价值(v)	价值/重量(v/w)
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

用于计算上下界

背包容量 $C = 10$

物品	重量(w)	价值(v)	价值/重量(v/w)
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

下界 (lower bound, lb) : 应用贪心法, 在背包容量 $C = 10$ 约束下的解为 $(1, 0, 1, 0)$, 获得的价值为 $40+25=65$, 可以作为0-1背包问题的下界。

背包容量 $C = 10$

物品	重量(w)	价值(v)	价值/重量(v/w)
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

目标函数的界

[65, 100]

下界 (lower bound, lb) : 应用贪心法, 在背包容量约束下的解为(1, 0, 1, 0), 上界 (upper bound, ub) : 应用贪心法, 在背包容量约束下的解为(1, 1, 0, 0), 即

上界 (upper bound, ub) :

上背包剩余容量

收紧上界 (缩小ub值), 直至最优解

$$ub = v + (C - w) * v_i / w_i$$

$$\text{初始值为: } ub = 0 + (10 - 0) * 10 = 100$$

物品	重量(w)	价值(v)	价值/重量(v/w)
 1	4	40	10
 2	7	42	6
3	5	25	5
4	3	12	4

放入物品1

$$ub = 40 + (10 - 4) * 6 = 76$$

放入物品2，总重量为
11 > 10，无效解

不放入物品2，

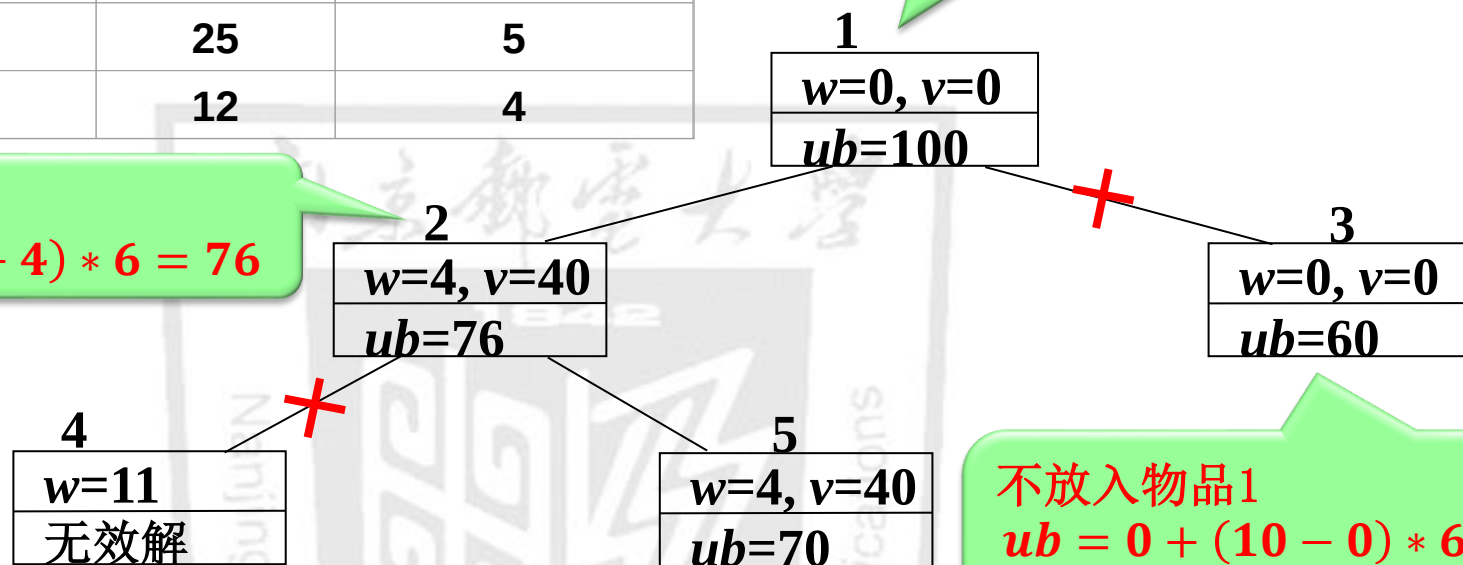
$$ub = 40 + (10 - 4) * 5 = 70$$

无物品放入

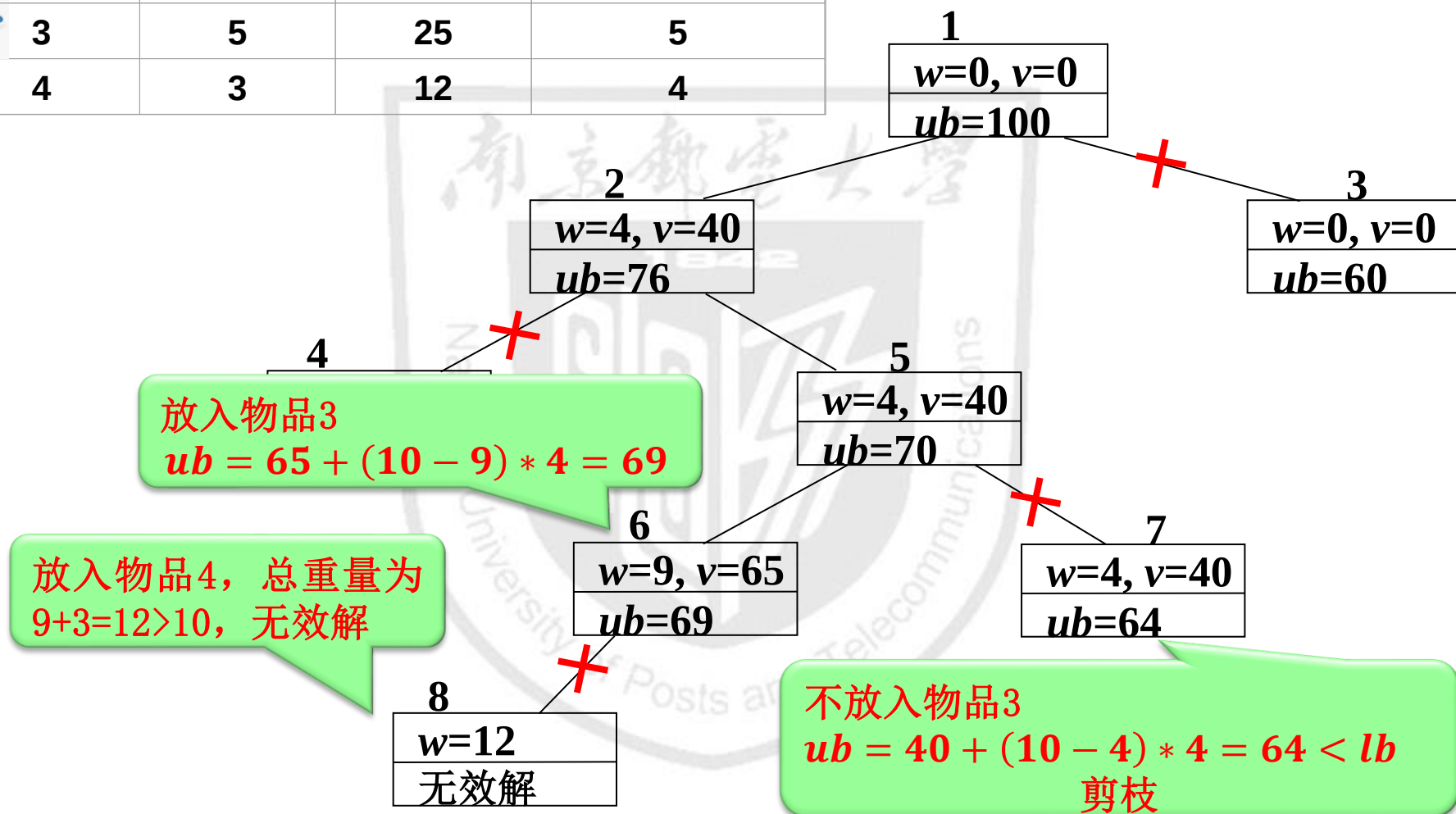
不放入物品1

$$ub = 0 + (10 - 0) * 6 = 60 < lb$$

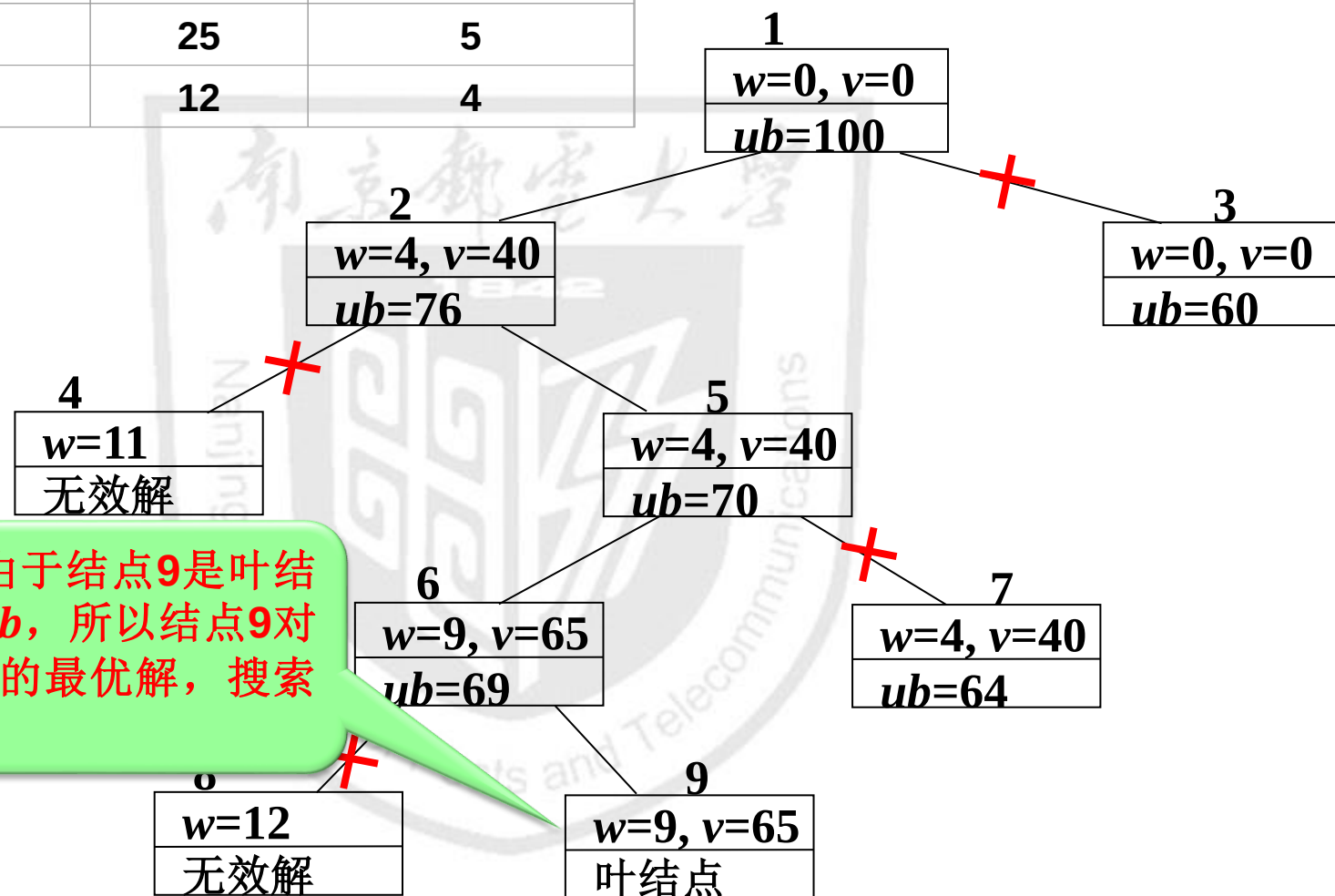
剪枝



物品	重量(w)	价值(v)	价值/重量(v/w)
✓ 1	4	40	10
✗ 2	7	42	6
✓ 3	5	25	5
4	3	12	4



	物品	重量(w)	价值(v)	价值/重量(v/w)
✓	1	4	40	10
✗	2	7	42	6
✓	3	5	25	5
✗	4	3	12	4



不放入物品4。由于结点9是叶结点，同时 $65 \geq ub$ ，所以结点9对应的解即是问题的最优解，搜索结束。

分支限界法

分支限界算法设计通常包含以下2个主要步骤：

1. 针对所给问题，定义问题的状态空间图
2. 以**广度优先**方式搜索状态空间图，并在搜索过程中用**剪枝策略**避免无效搜索

```
function BranchBound(problem, queue) {  
    node = Make-Node(Initial-State[problem]);  
    queue ← Insert(node, queue)  
    do while(1)  
        if queue == Empty  
            return failure  
        node ← Remove-First(queue)  
        visit(node);  
        if State[node] == Goal  
            return Solution(node)  
        while (((sonNode = Next(problem, node)) != NULL) {  
            //子结点约束条件与限界条件判定  
            if (constraint(sonNode) && bound(sonNode) )  
                Insert(queue, sonNode);  
        }  
    } }
```

小试牛刀

假设在0-1背包问题中，有编号为 $\{1,2,3,4\}$ 的4个物品，其重量分别为 $w = \{4,7,6,3\}$ ，价值分别为 $v = \{32,42,30,9\}$ ，背包容量 $C = 10$ 。

试采用分支限界法解决该0-1背包问题。要求：

- (1) 给出背包物品总价值的合理上界；
- (2) 用贪心法选择合适初始下界；
- (3) 给出分支限界法求解该0-1背包问题的过程（要求体现解空间搜索特点），给出最优解。



(1) 已经装入背包中的物品取得的价值 V ，背包剩余容量为 $C - W$ ，剩下物品的最大单位重量价值 v_i/w_i ，则背包总价值上界为 $ub = V + (C - W) \times v_i/w_i$ 。

(2) 将物品按单位重量价值由大到小排列，可得

n	1	2	3	4
v_n/w_n	8	6	5	3

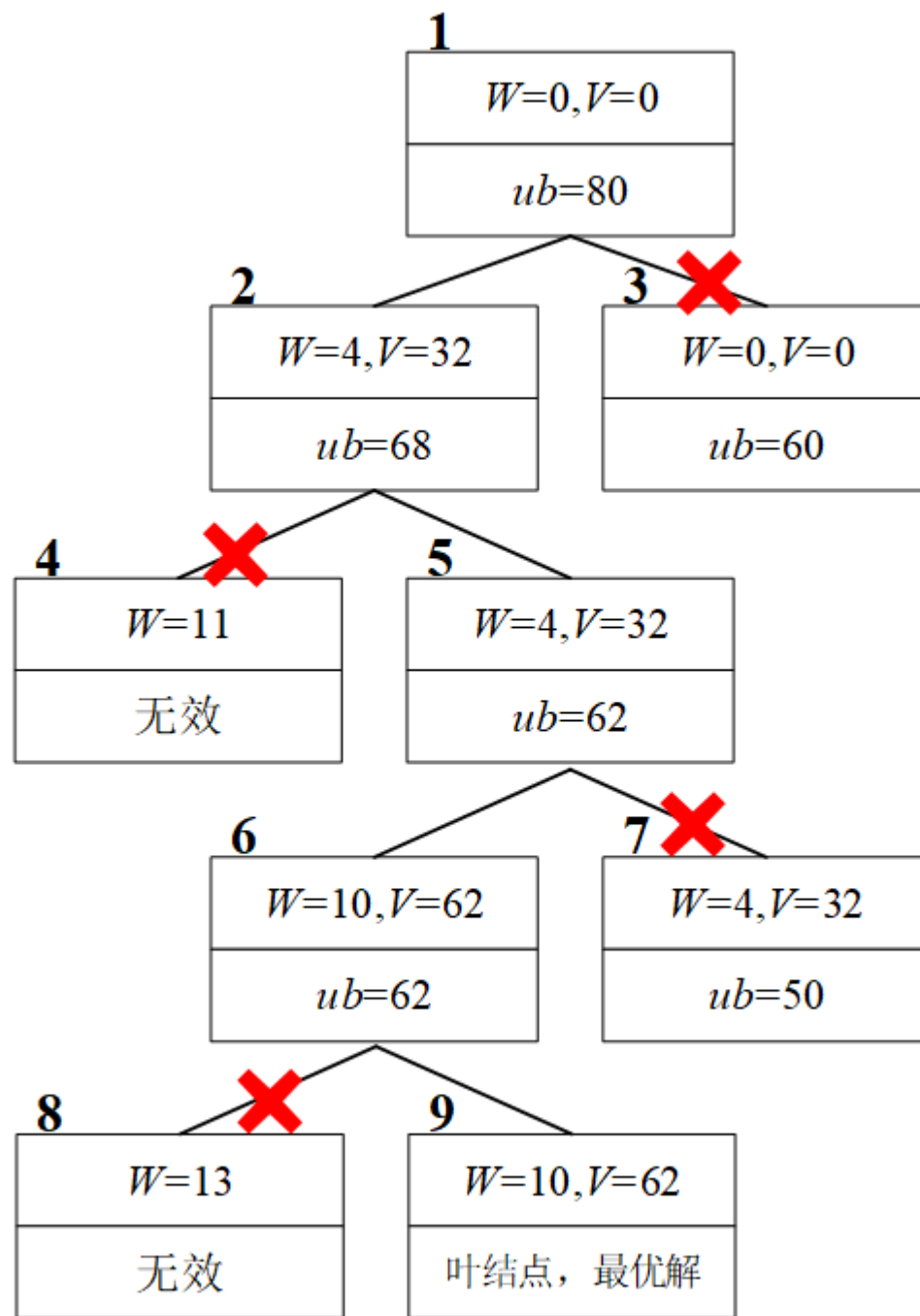
在背包容量容许的条件下，依次装入单位重量价值最大物品，可得可行解 $\{1,0,1,0\}$ ，相应背包物品价值为 62。

背包物品价值初始下界为 62，对应物品放入情况为 $\{1,0,1,0\}$ 。





(3)



代码实现

```
typedef struct Node {
    int level;           // 当前节点在搜索树中的层次
    int profit;          // 当前节点的总价值
    int weight;          // 当前节点的总重量
    double bound;        // 当前节点的价值上界
    struct Node* parent; // 父节点指针
} Node;

// 计算价值上界
double bound(Node* u, int w[], int p[], int C) {
    if (u->weight >= C) {
        return 0;
    }
    double b = u->profit;
    int j = u->level + 1;
    int totweight = u->weight;
    while (j < N && totweight + w[j] <= C) {
        totweight += w[j];
        b += p[j];
        j++;
    }
    if (j < N) {
        b += (C - totweight) * ((double)p[j] / w[j]);
    }
    return b;
}
```

分支限界法与回溯法的异同

一、分支限界法与回溯法的共同点

都是在问题的状态空间树上搜索问题解的算法。

都用活结点表实现,都可用约束函数剪去不含答案结点的分支,都可用限界函数剪去不含最优解的分支。

二、分支限界法与回溯法的区别

- (1) **求解目标不同**: 回溯法的求解目标是找出解空间树中满足约束条件的**所有可行解**; 而分支限界法的求解目标则是找出满足约束条件的**一个可行解**, 或某种意义下的**最优解**。
- (2) **搜索方式不同**: 回溯法以**深度优先**的方式搜索解空间树, 而分支限界法则以**广度优先**或**最小耗费优先**的方式搜索解空间。
- (3) **对当前扩展结点的扩展方式不同**: 回溯法中的每个活结点可能**多次成为当前扩展结点**, 纵深方向扩其一个儿子, 然后再回溯后扩展其他儿子; 而分支限界法中每一个活结点只有**一次机会成为扩展结点**, 一次产生所有孩子结点, 自身成为死结点, 之后无需再返回该结点处。

常见的两种分支限界法

(1) 队列式(FIFO)分支限界法

- 按照队列**先进先出 (FIFO) 原则**选取下一个节点为扩展节点。
- 活节点表是FIFO队列。

(2) 优先队列式分支限界法

- 按照优先队列中规定的**优先级选取优先级最高的节点**成为当前扩展节点。
- 活节点表是优先权队列。

分支限界法和回溯法都是在问题的解空间树上搜索问题的解，两者的_____。

- ☐ A 求解目标不同，搜索方式相同
- ☒ B 求解目标不同，搜索方式也不同
- ☐ C 求解目标相同，搜索方式不同
- ☐ D 求解目标相同，搜索方式也相同

在采用优先队列式分支限界法求解0/1背包问题时，活结点表的组织形式是_____。

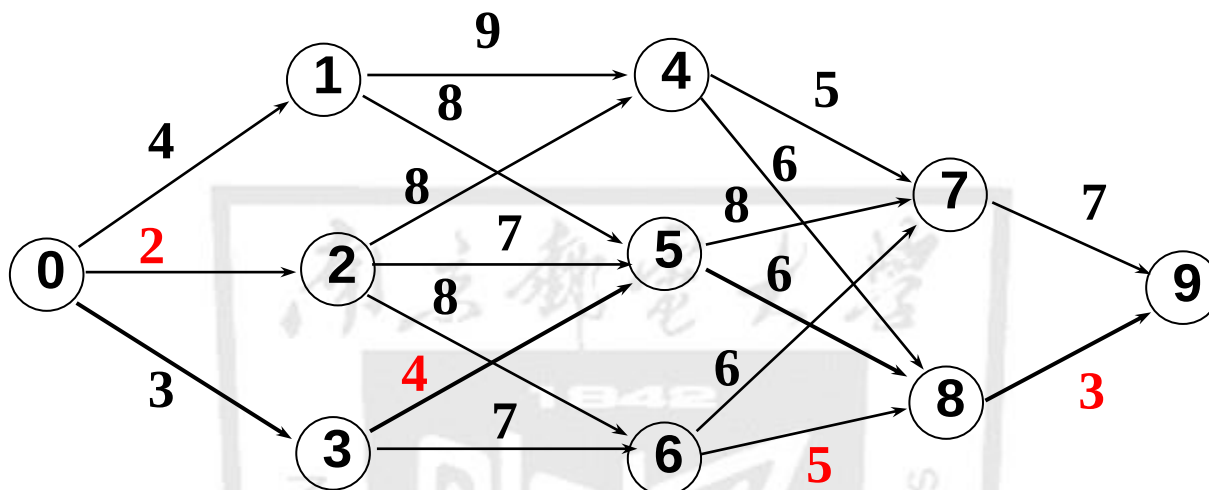
- ☐ A 最小堆
- ☐ B 数组
- ☐ C 栈
- ☒ D 最大堆



常见的两种分支限界法是_____。

- ☐ A 广度优先分支限界法与深度优先分支限界法
- ☐ B 队列式分支限界法与栈式分支限界法
- ☒ C 队列式分支限界法与优先队列分支限界法
- ☐ D 排列数法与子集树法

单源最短路径问题



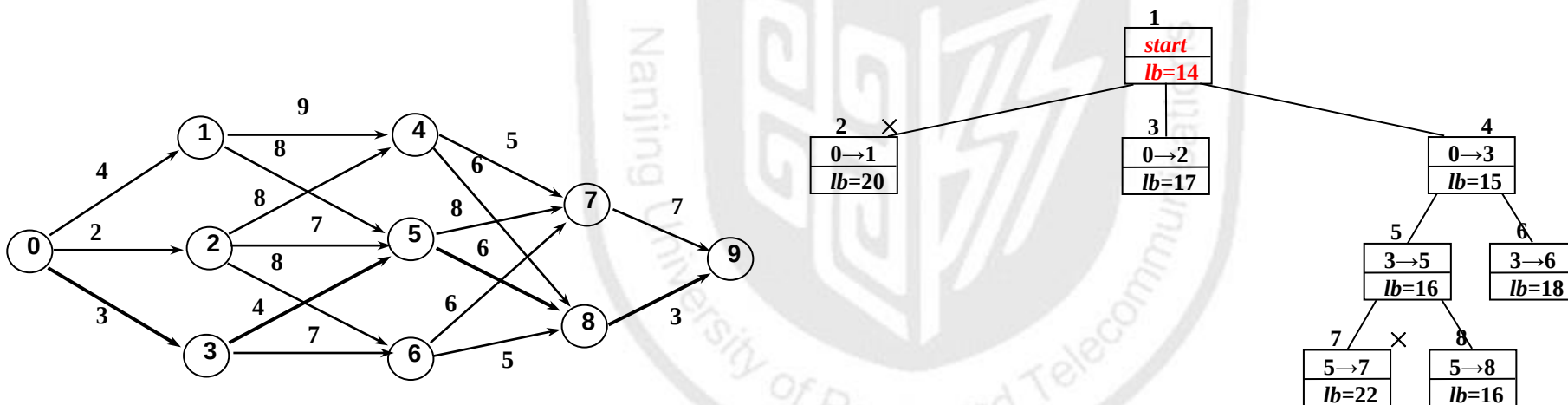
上界

应用贪心法求得近似解为 $0 \rightarrow 2 \rightarrow 5 \rightarrow 8 \rightarrow 9$ ，其路径代价为 $2+7+6+3=18$ ，可作为最短路径问题的上界。

下界

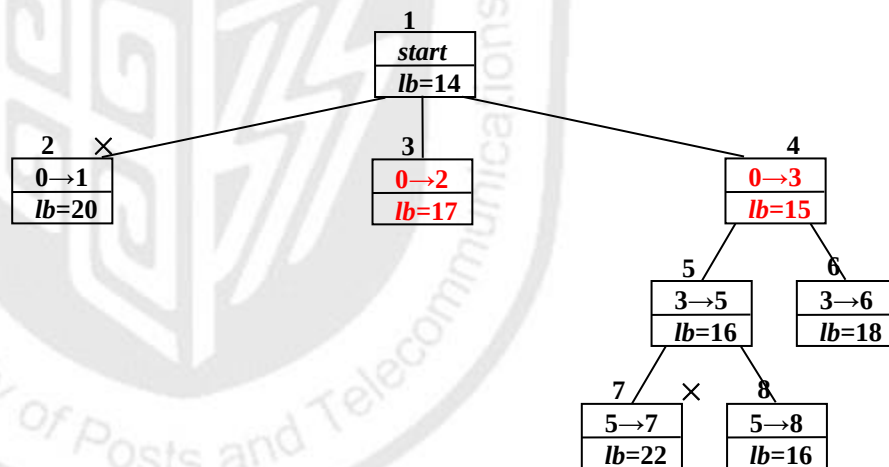
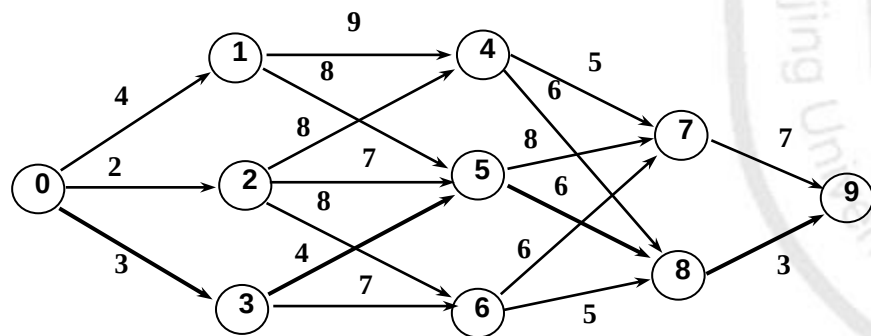
把每一段最小的代价相加，可以得到一个非常简单的下界，其路径长度为 $2+4+5+3=14$ 。

- 在根结点0，根据限界函数计算目标函数的值为14；
- 在结点1，第1段选择边<0, 1>，目标函数值为 $lb=4+8+5+3=20$ ，超出目标函数的界，将结点1丢弃；
 - (1) 0-1已选中，即第1和第2段已标记，故4（0-1）+ 8（1的边中选择最小）
 - (2) 第3和第4段未标记，选取每段最小边相加，即 5+3

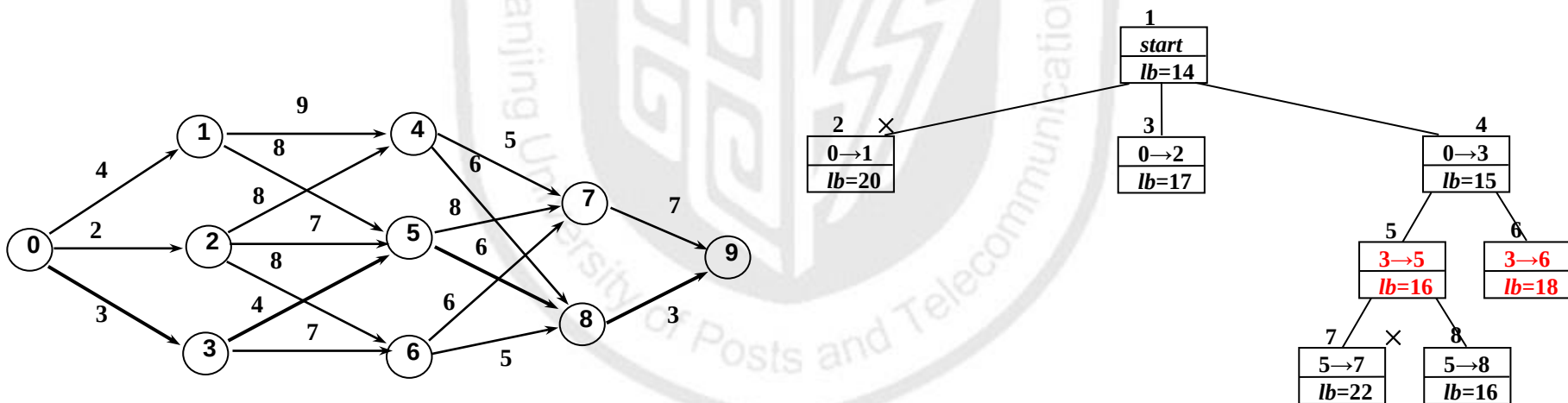




- 在结点2，第1段选择边<0, 2>，目标函数值为 $lb=2+7+5+3=17$ ，将结点2加入队列中；
- 在结点3，第1段选择边<0, 3>，目标函数值为 $lb=3+4+5+3=15$ ，将结点3加入队列中；
- 在队列中选取目标函数值极小的结点3优先进行搜索；

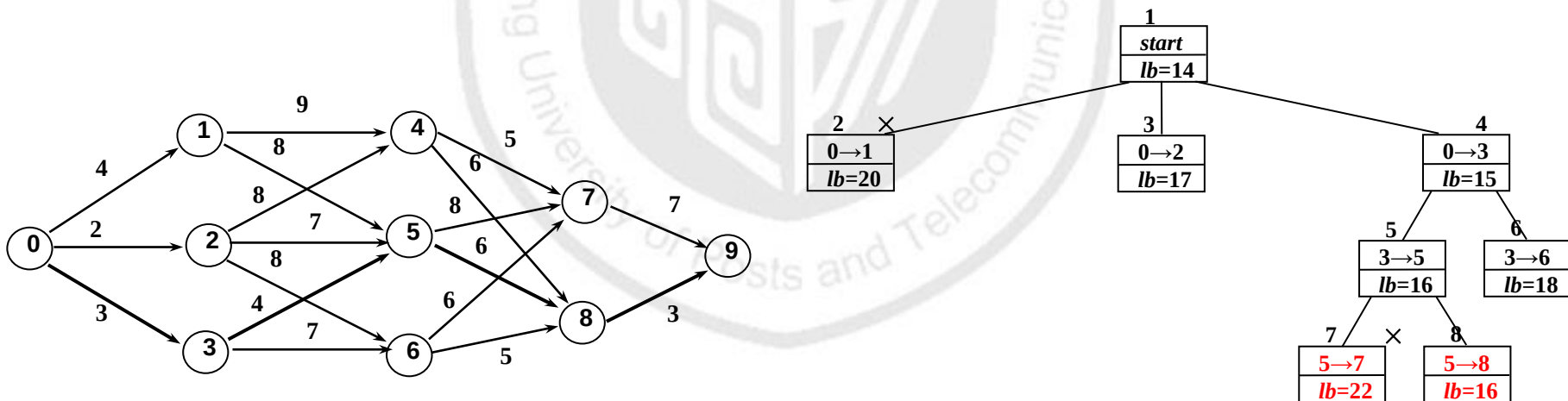


- 在结点5，第2段选择边<3, 5>，目标函数值为 $lb=3+4+6+3=16$ ，将结点5加入队列中；
- 在结点6，第2段选择边<3, 6>，目标函数值为 $lb=3+7+5+3=18$ ，将结点6加入队列中；
- 在队列中选取目标函数值极小的结点5优先进行搜索；



- 在结点7，第3段选择边<5, 7>，可直接确定第4段的边<7, 9>，目标函数值为 $lb=3+4+8+7=22$ ，为一个可行解但超出目标函数的界，将其丢弃；
- 在结点8，第3段选择边<5, 8>，可直接确定第4段的边<8, 9>，目标函数值为 $lb=3+4+6+3=16$ ，为一个较好的可行解。
- 由于结点8是叶子结点，并且其目标函数值是队列中最小的，所以，结点8代表的解即是问题的最优解，搜索过程结束。

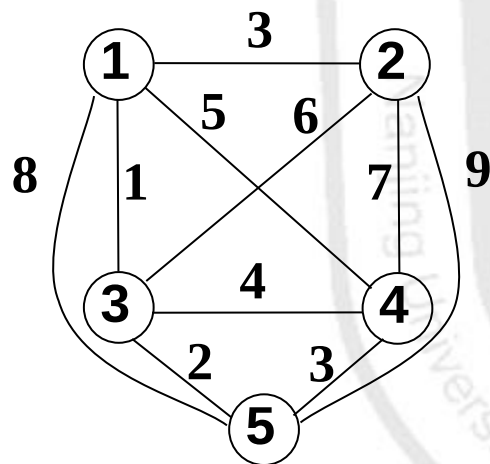
路径：0 → 3 → 5 → 8 → 9





旅行售货员问题

问题描述：某售货员要到若干城市去推销商品，已知各城市之间的路程（或旅费）。他要选定一条从驻地出发，**经过每个城市一次，最后回到驻地**的路线，使**总的路程(或总旅费)最小**。



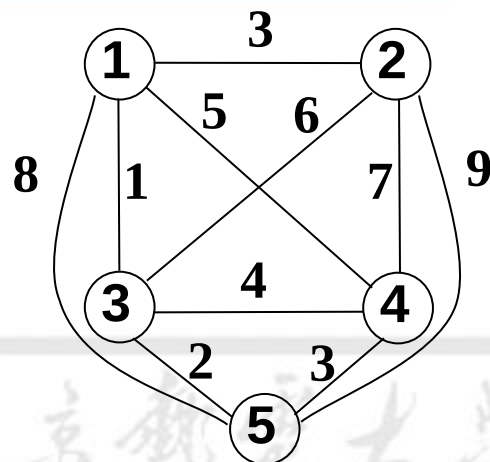
(a) 一个无向图

$$C = \begin{bmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{bmatrix}$$

(b) 无向图的代价矩阵

上界

采用贪心法求得近似解为
 $1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ，其路径长度为 $1+2+3+7+3=16$ ，作为TSP问题的上界。


$$C = \begin{bmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{bmatrix}$$

下界

2种方法

- 把矩阵中
- 考虑一个
- 接边，一
- 的两个元素相加

收紧下界（缩小lb值），
直至最优解

其路

两条邻
行最小

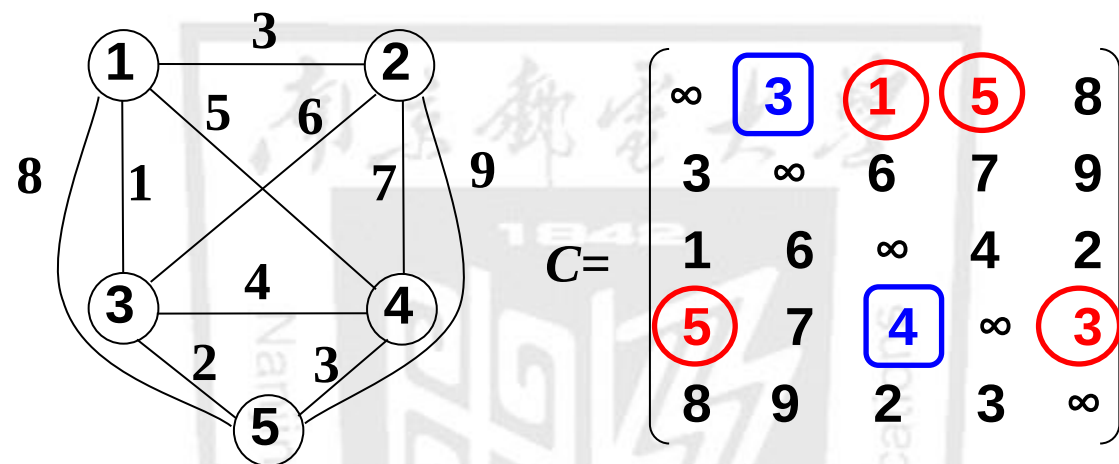
对最后

来向上取整，可得到一个合理的下界：

$$lb = [(1+3) + (3+6) + (1+2) + (3+4) + (2+3)] / 2 = 14$$

需要强调的是，该解并不一定是有效选择（可能没有构成哈密顿回路），它仅仅给出了一个参考下界。

部分解的目标函数值的计算方法



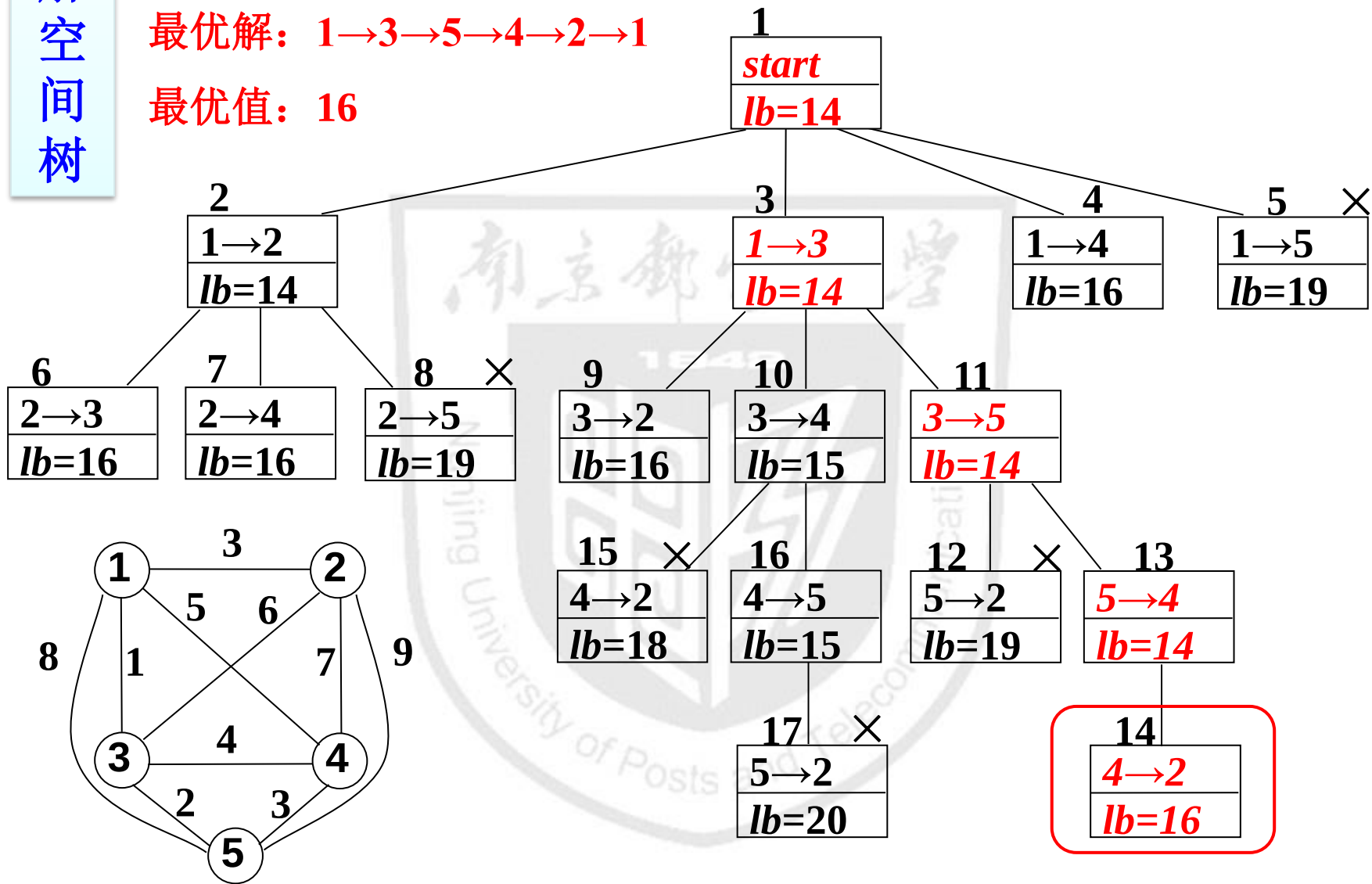
如果部分解包含边(1, 4), 则该部分解的下界是

$$lb = ((1+5) + (3+6) + (1+2) + (3+5) + (2+3)) / 2 = 16。$$

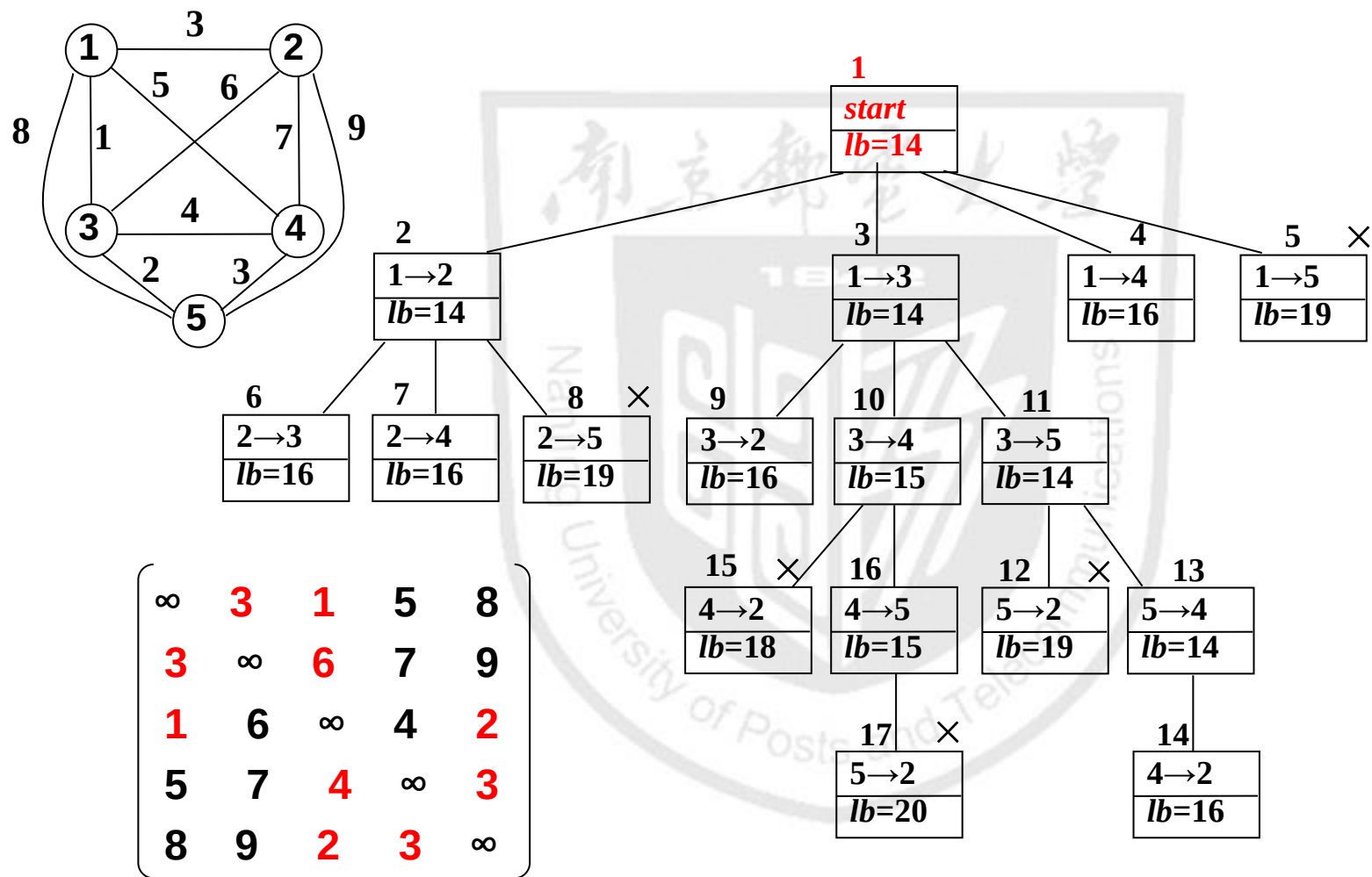
解空间树

最优解: 1→3→5→4→2→1

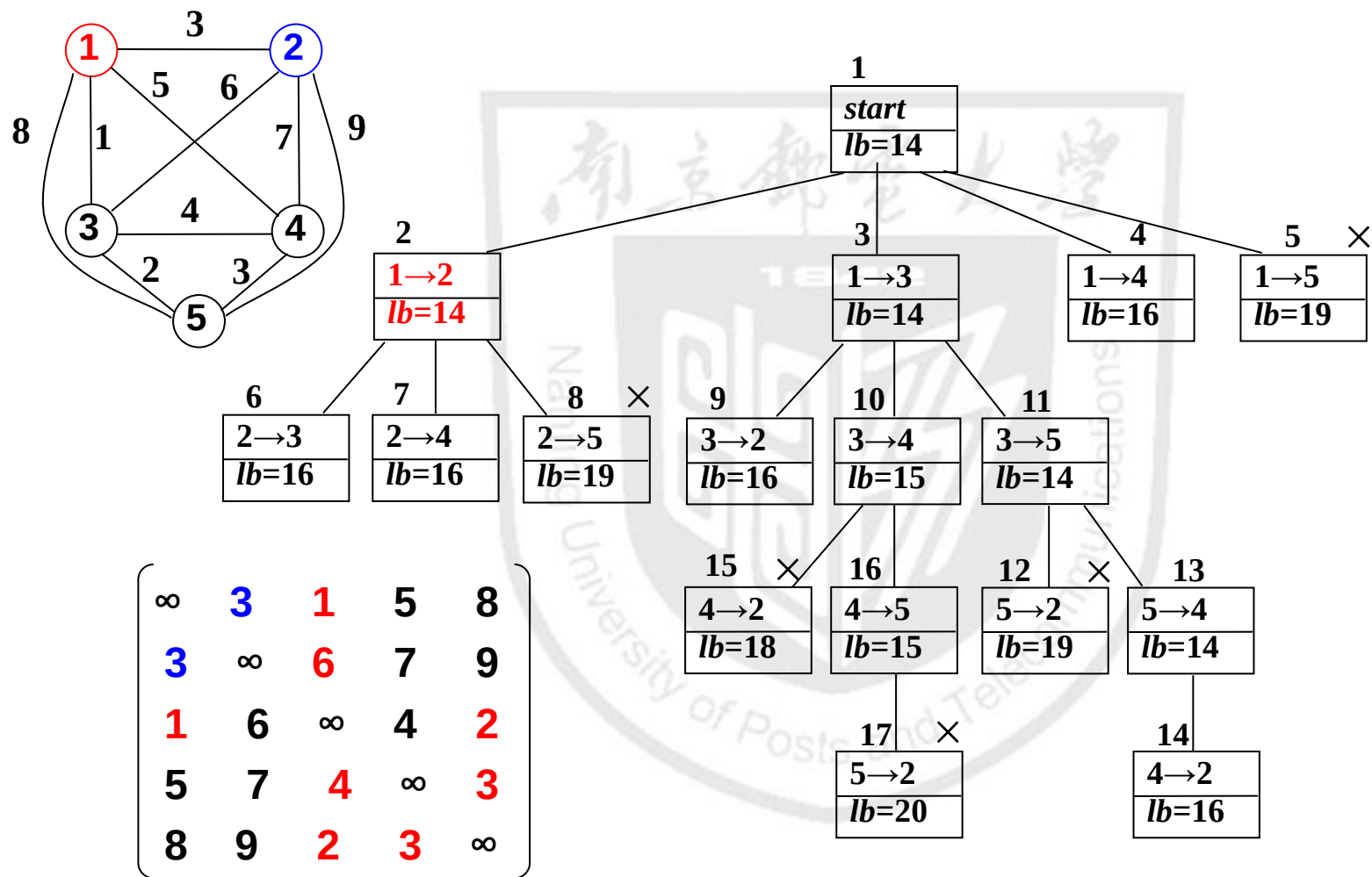
最优值： 16



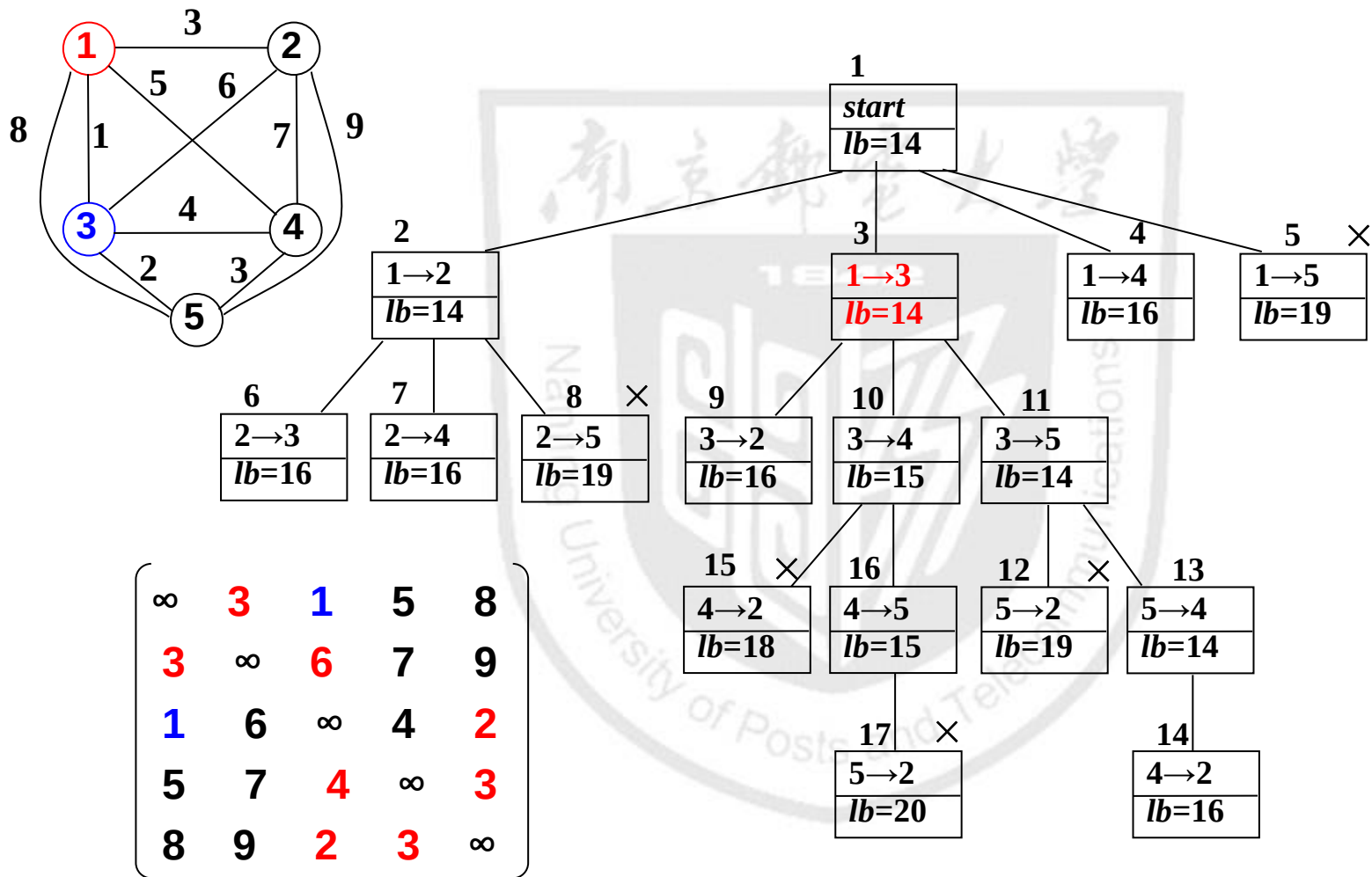
➤ 在根结点1，根据限界函数计算目标函数的值为
 $lb=((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=14;$



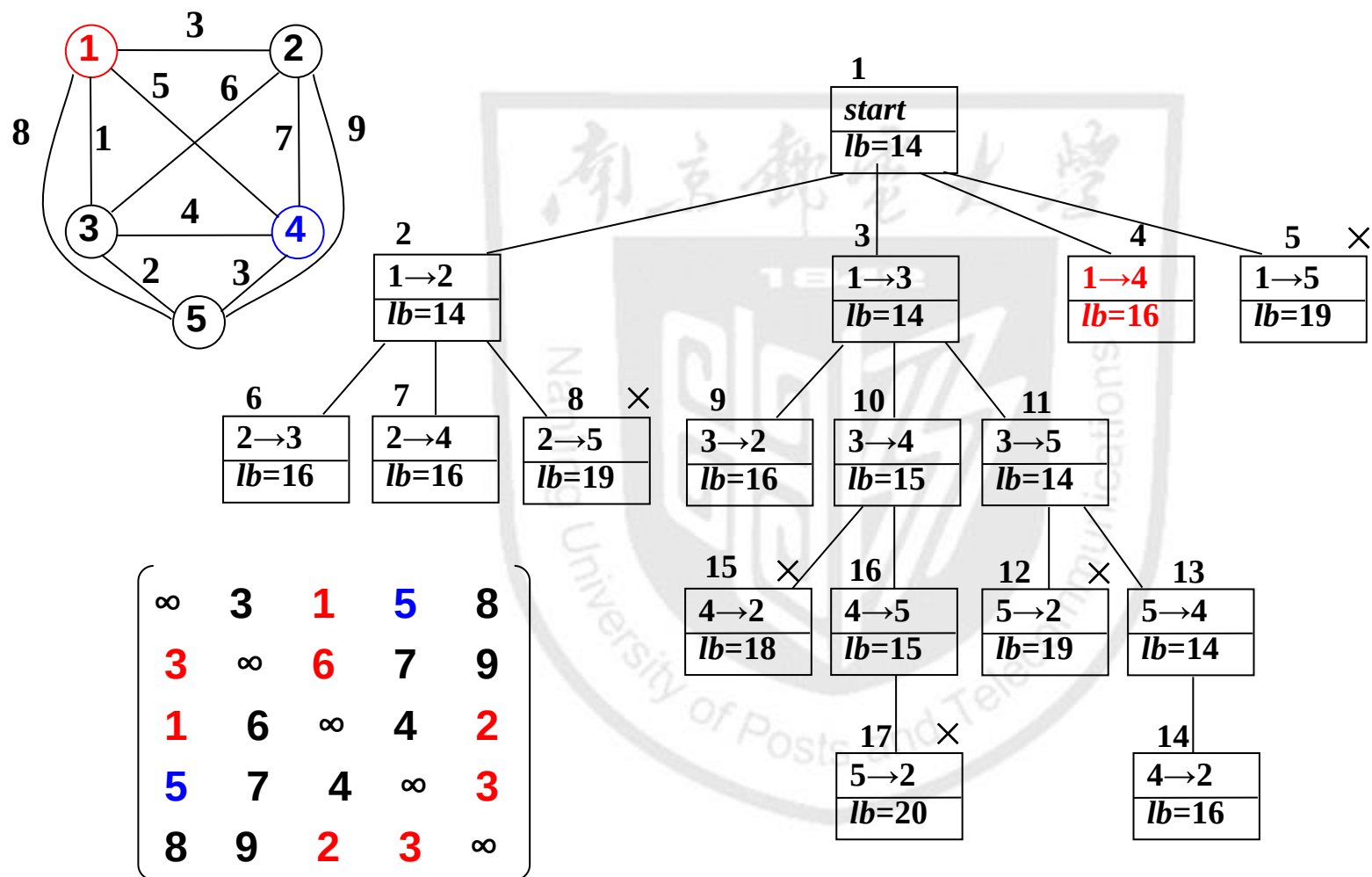
- 在结点2，从城市1到城市2，路径长度为3，目标函数的值为 $((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=14$ ，将结点2加入队列；



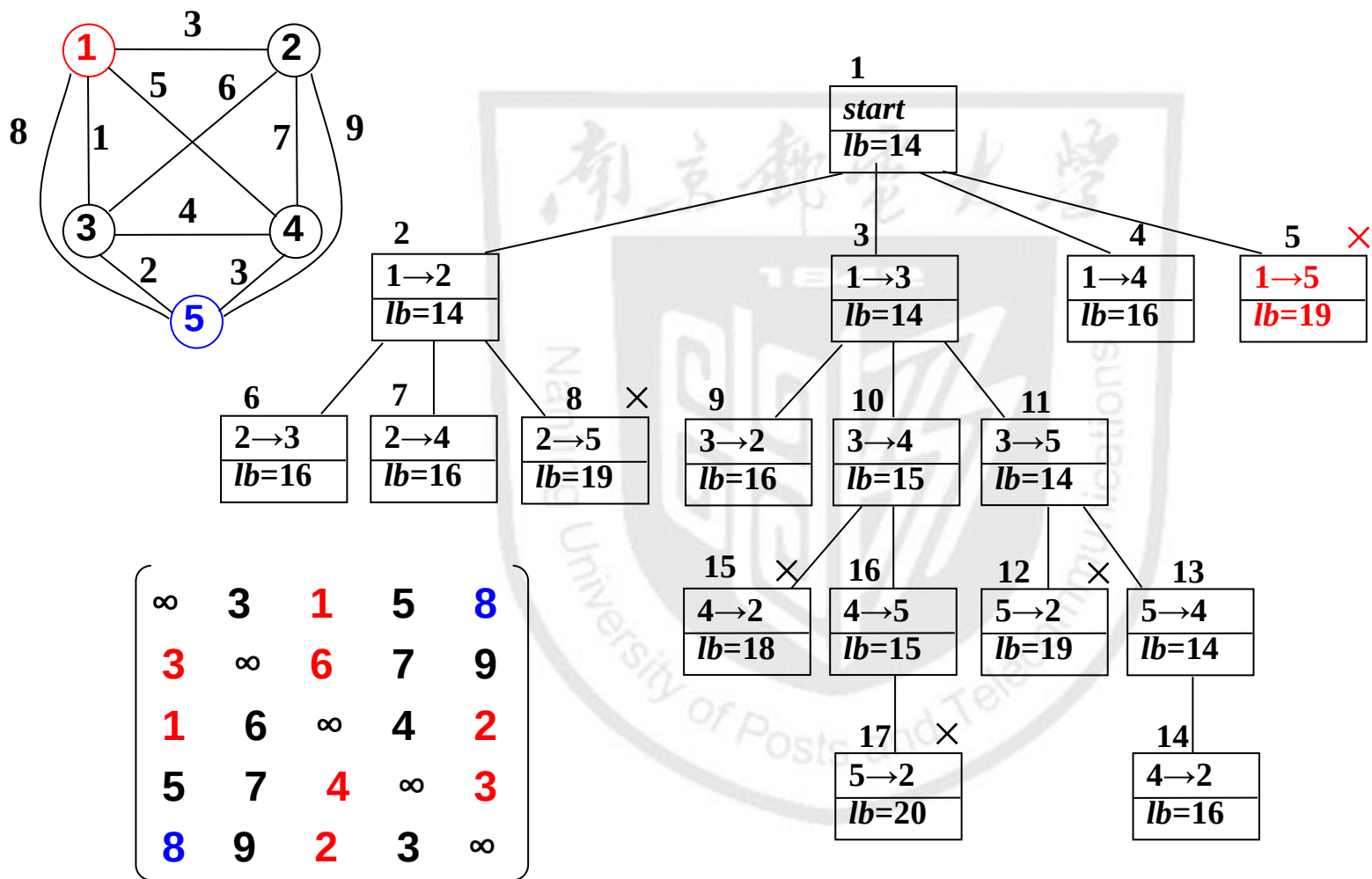
- 在结点3，从城市1到城市3，路径长度为1，目标函数的值为 $((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=14$ ，将结点3加入队列；



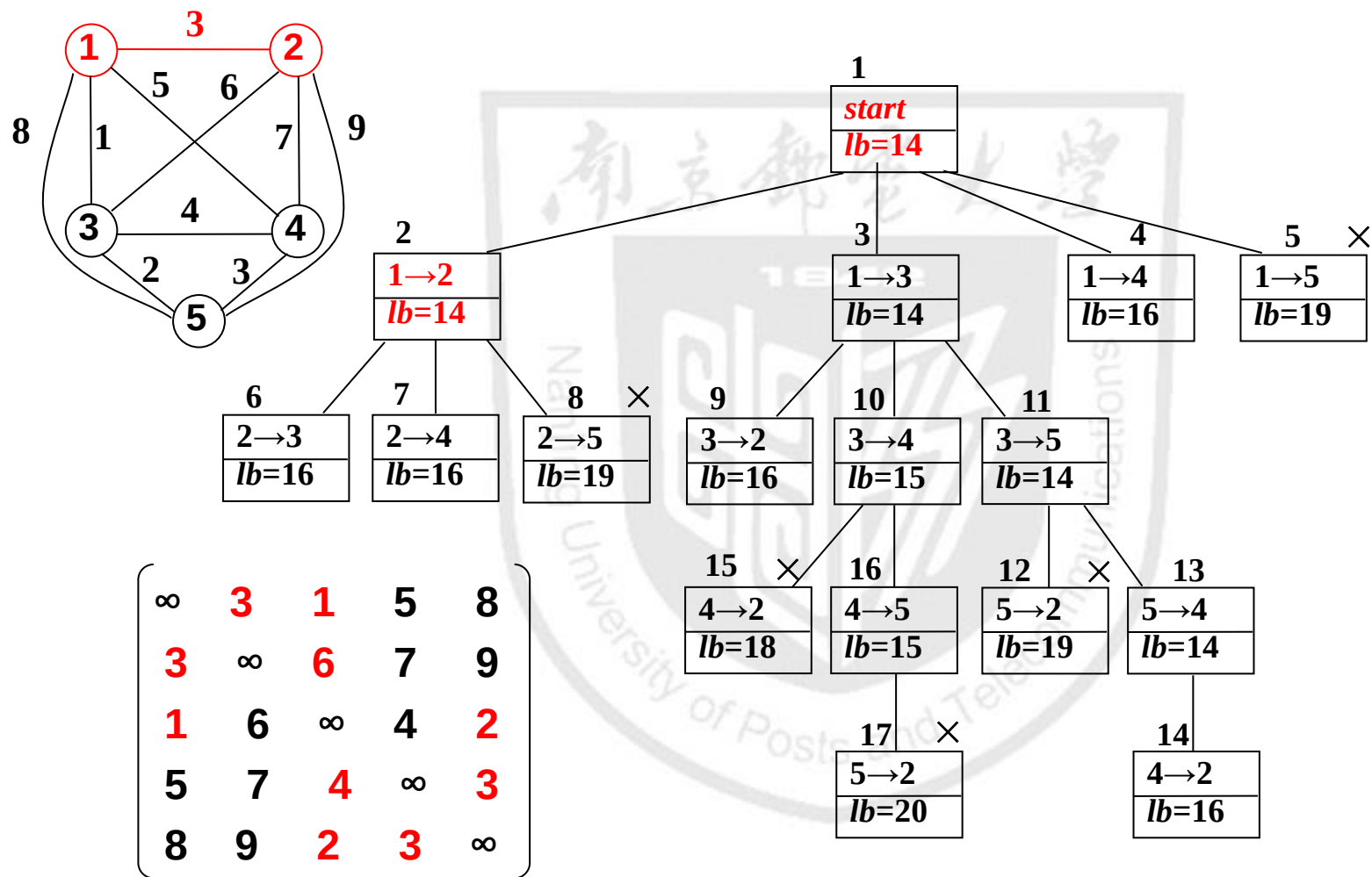
- 在结点4，从城市1到城市4，路径长度为5，目标函数的值为 $((1+5)+(3+6)+(1+2)+(3+5)+(2+3))/2=16$ ，将结点4加入队列；



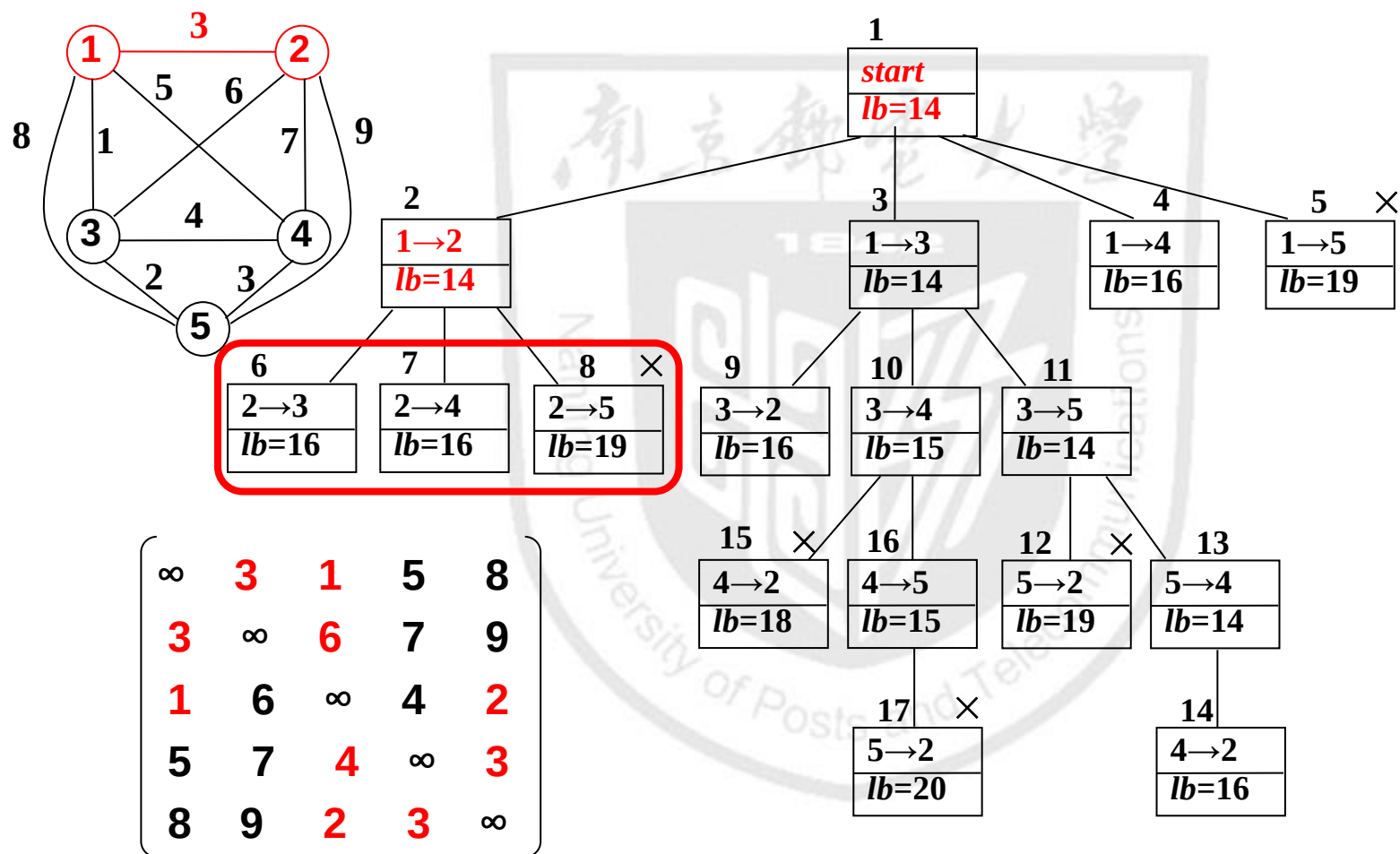
- 在结点5，从城市1到城市5，路径长度为8，目标函数的值为 $((1+8)+(3+6)+(1+2)+(3+4)+(2+8))/2=19$ ，超出目标函数的上界 $up=16$ ，将结点5丢弃；

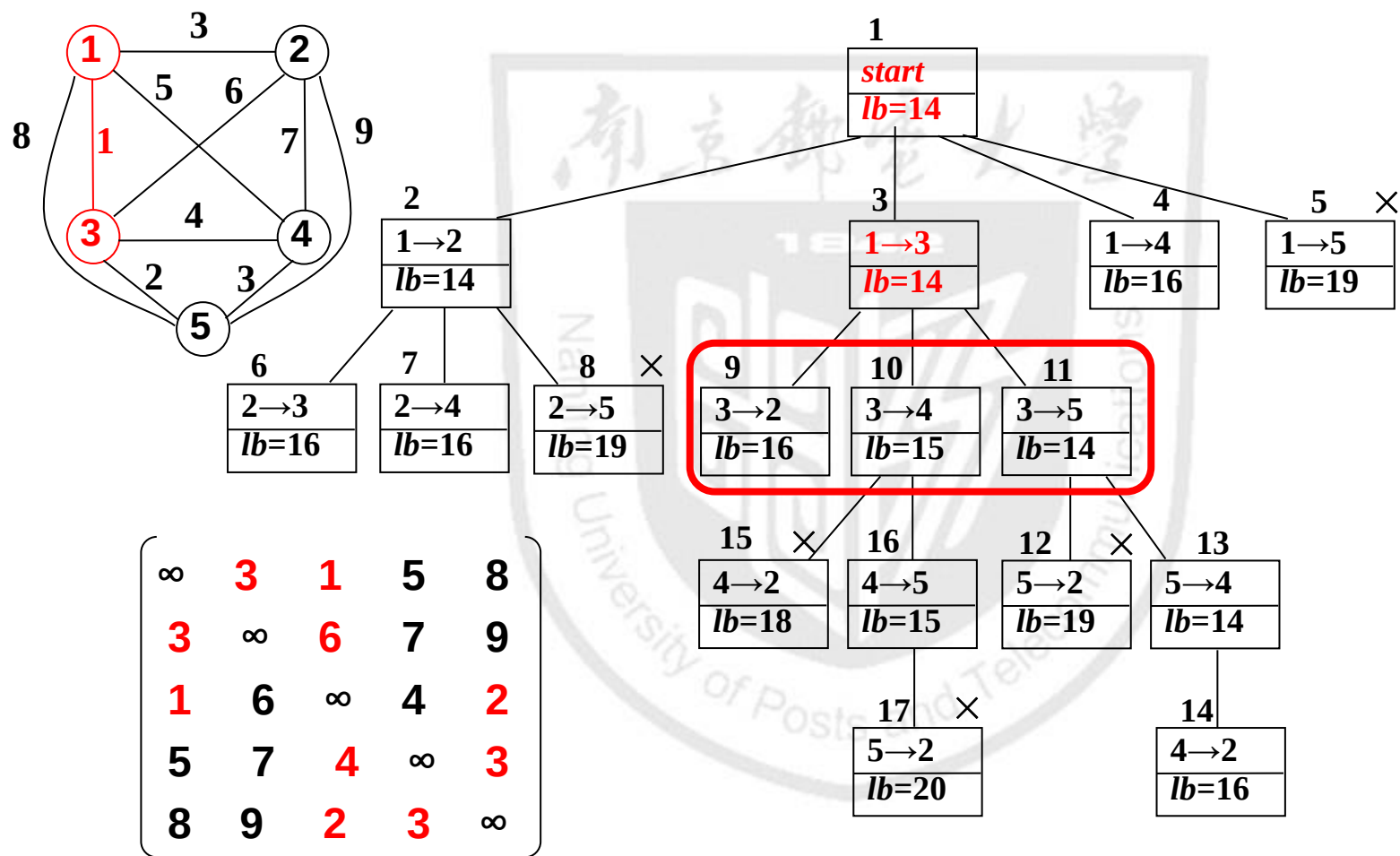
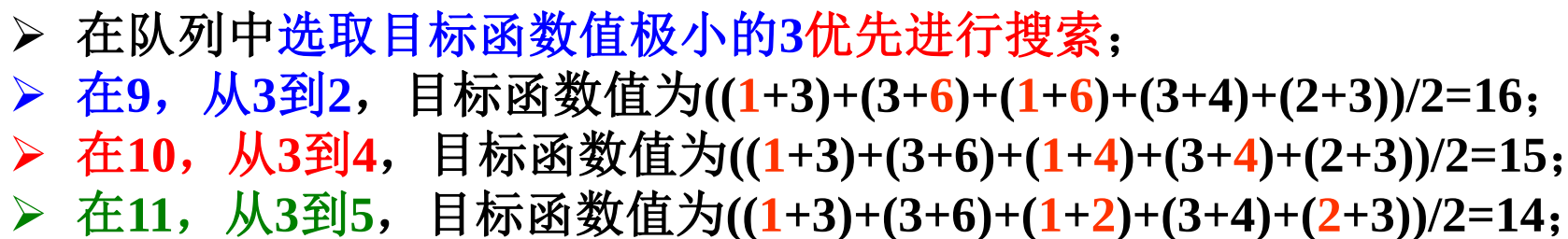


➤ 在队列中选取目标函数值极小的**结点2**优先进行搜索；

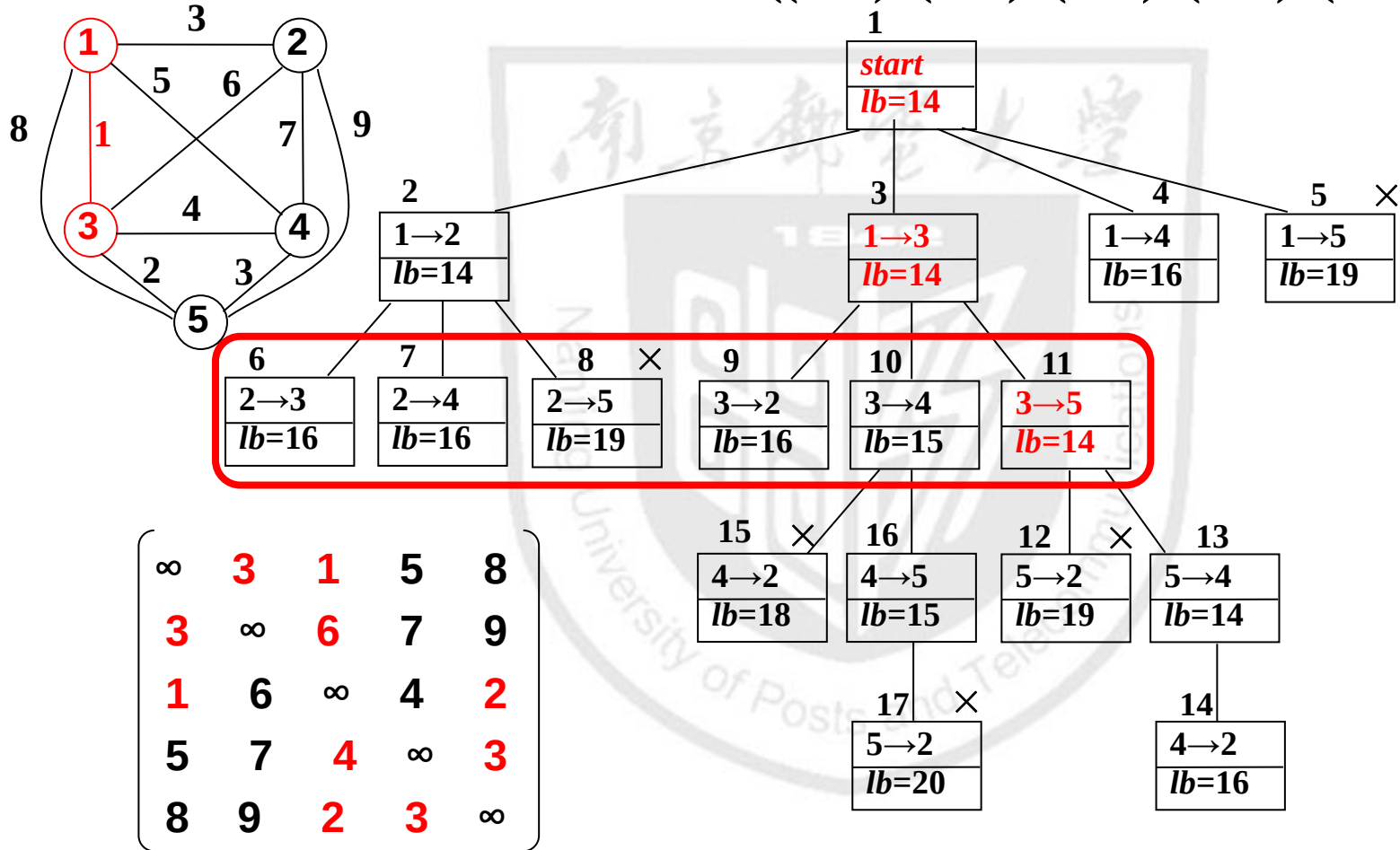


- 在6, 从2到3, 目标函数值为 $((1+3)+(3+6)+(1+6)+(3+4)+(2+3))/2=16$;
- 在7, 从2到4, 目标函数值为 $((1+3)+(3+7)+(1+2)+(3+7)+(2+3))/2=16$;
- 在8, 从2到5, 目标函数值为 $((1+3)+(3+9)+(1+2)+(3+4)+(2+9))/2=19$, 超出目标函数的界, 将结点8丢弃;

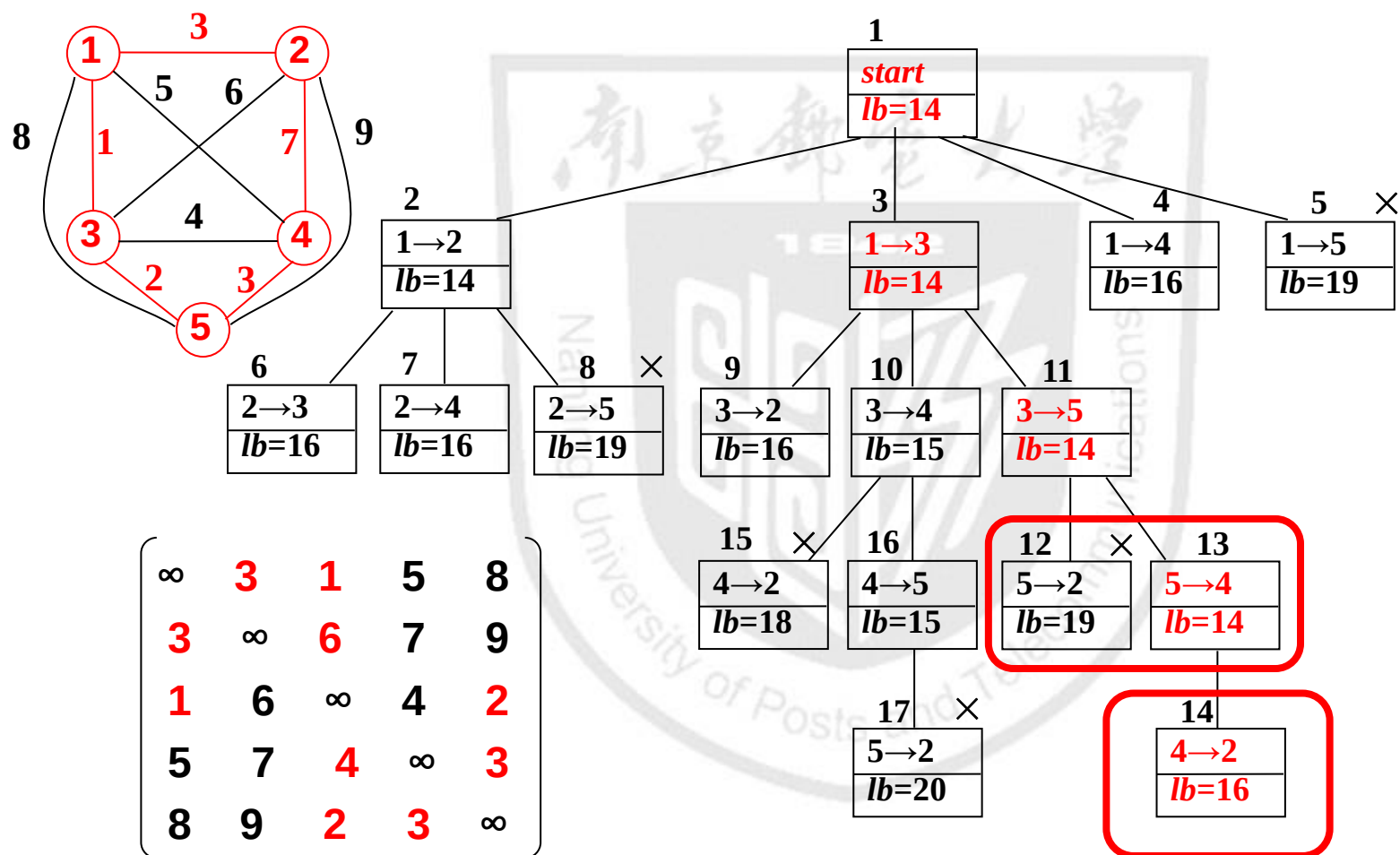




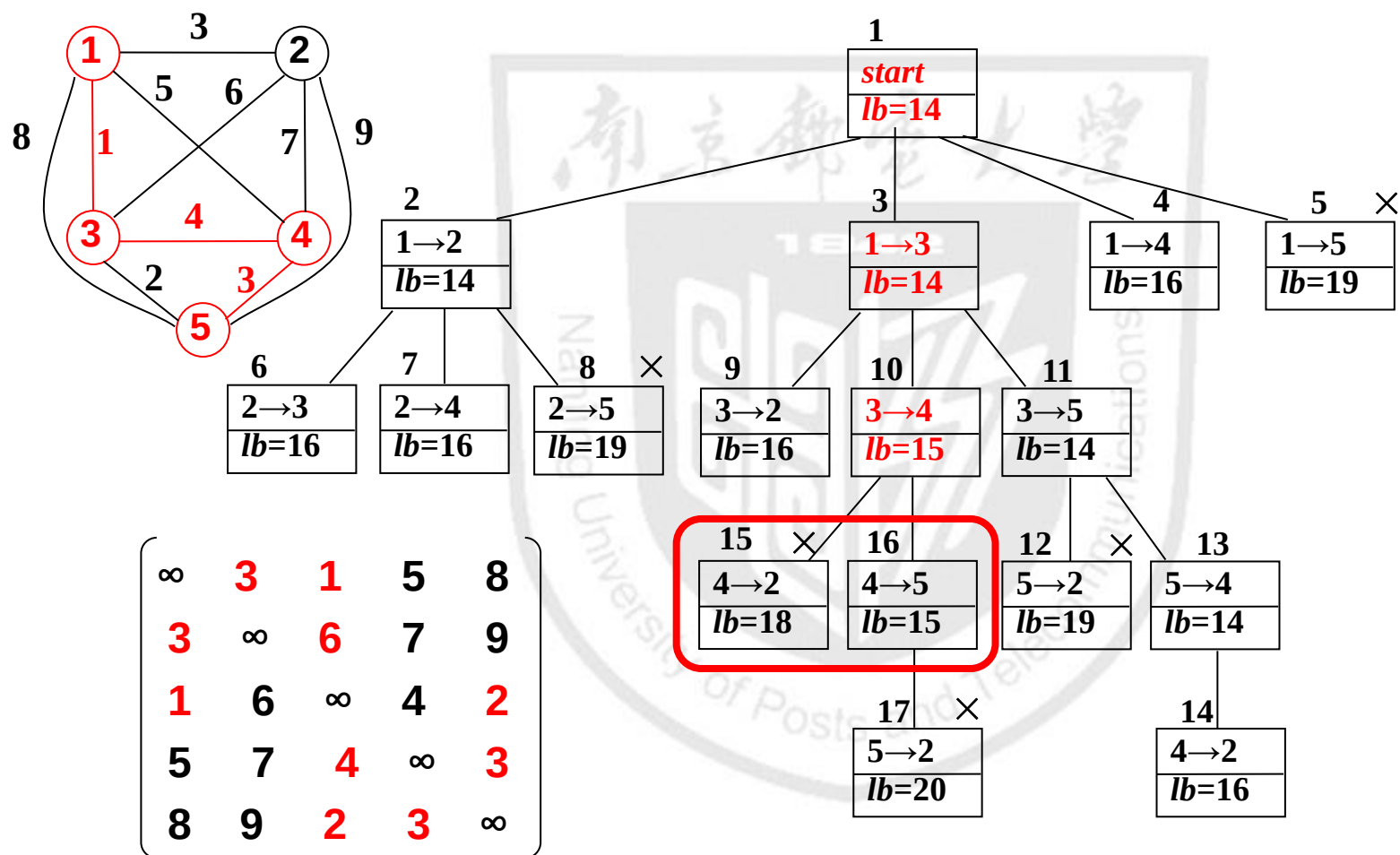
- 在队列中选取目标函数值极小的结点11优先进行搜索；
- 在12，从5到2，目标函数值为 $((1+3)+(3+9)+(1+2)+(3+4)+(2+9))/2=19$ ，超出目标函数的界，将结点12丢弃；
- 在13，从5到4，目标函数值为 $((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=14$ ；



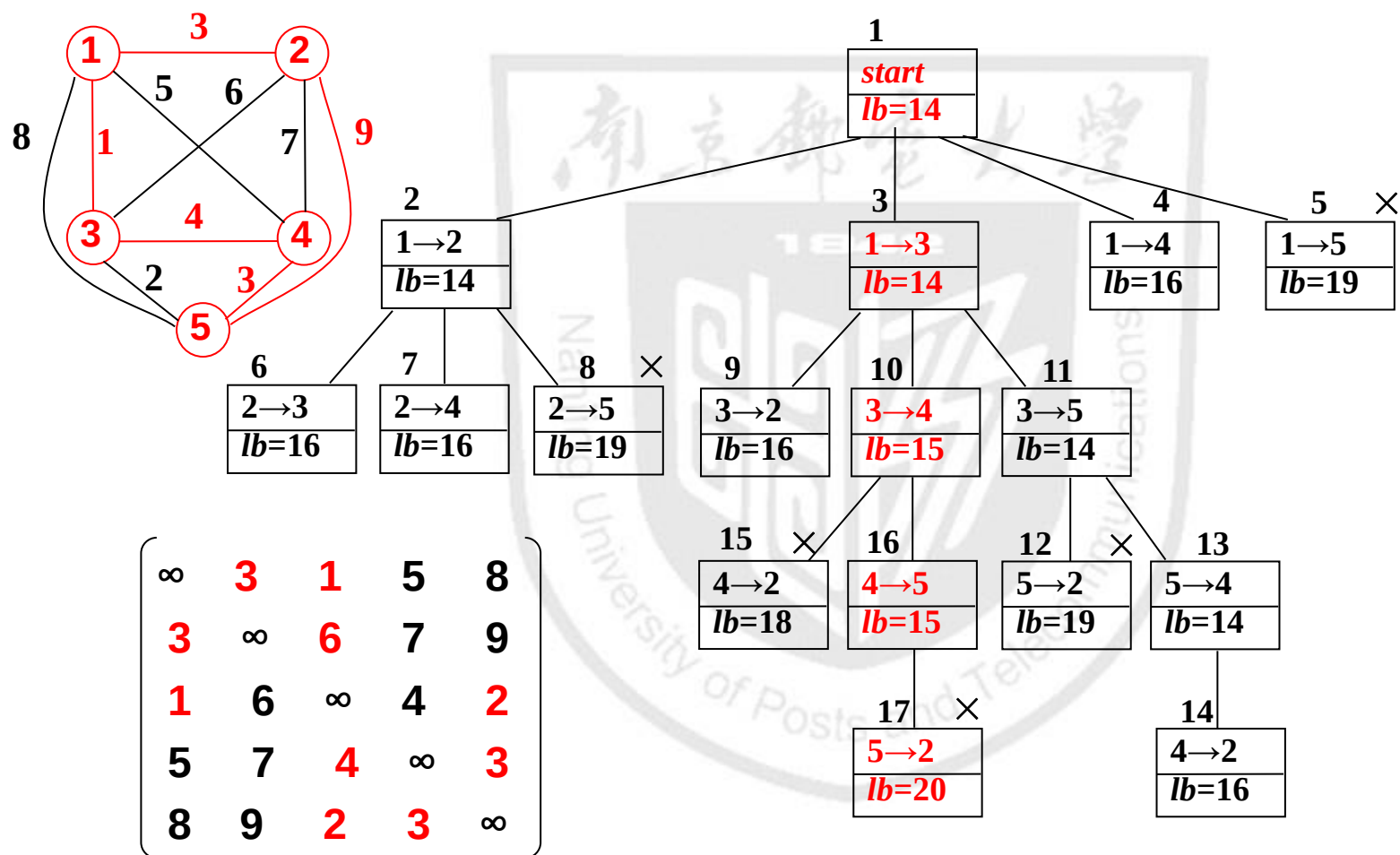
- 在队列中选取目标函数值极小的结点13优先进行搜索；
- 在14，从4到2，目标函数值为 $((1+3)+(3+7)+(1+2)+(3+7)+(2+3))/2=16$ ；
- 最后从2回到1，目标函数值为 $((1+3)+(3+7)+(1+2)+(3+7)+(2+3))/2=16$ ，由于14为叶子结点，得到一个可行解，其路径长度为16；



- 在队列中选取目标函数值极小的结点10优先进行搜索；
- 在15，从4到2，目标函数的值为 $((1+3)+(3+7)+(1+4)+(7+4)+(2+3))/2=18$ ，超出目标函数的界，将结点15丢弃；
- 在16，从4到5，目标函数值为 $((1+3)+(3+6)+(1+4)+(3+4)+(2+3))/2=15$ ；



- 在列表选取目标函数值极小的结点16优先进行搜索
- 在17，从5到2，目标函数的值为 $((1+3)+(3+9)+(1+4)+(3+4)+(9+3))/2=20$ ，超出目标函数的界，将结点17丢弃；
- 目标函数值均为16，且有一个是叶子16。所以，16对应的解 $1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 即是TSP问题的最优解，搜索过程结束。



在采用优先队列式分支限界法求解旅行售货员问题时，活结点表的组织形式是_____。

- ☒ A 最小堆
- ☐ B 数组
- ☐ C 栈
- ☐ D 最大堆



装载问题

问题描述：有 n 个集装箱要装上2艘载重量分别为 C_1 和 C_2 的轮船，其中集装箱 i 的重量为 W_i ，且 $\sum W_i \leq C_1 + C_2$ 。请设计算法确定是否有一个合理的装载方案可将这些集装箱上这2艘轮船。

输入：多组测试数据。每组测试数据包括两行：第一行输入集装箱数目 n ($n < 1000$)，以及两艘轮船的载重 C_1 和 C_2 ；第二行输入 n 个整数，表示每个集装箱的重量。

输出：如果存在合理装载方案，输出第一艘轮船的最大装载重量；否则，输出“No”。

输入样例：

3 50 50

10 40 40

3 50 50

20 40 40

输出样例

50

No

把原问题简化为一艘轮船的装载问题。容易证明下述结论：如果一个给定装载问题实例有解，则采用下面的策略可得到最优装载方案：

- (1)首先将第一艘轮船**尽可能装满**；
- (2)将剩余的集装箱装上第二艘轮船。

问题分析

装载问题等价于以下特殊的0-1背包问题：

$$\begin{aligned} \max \quad & \sum_{i=1}^n w_i x_i \\ \text{st.} \quad & \begin{cases} \sum_{i=1}^n w_i x_i \leq C_1 \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

问题分析

1. 问题的状态表示

$$X = [wt, idx]$$

$$X = [x_1, x_2, \dots, x_k]$$

2. 约束条件

$$wt \leq C_1$$

3. 操作符

给定状态 $X = [wt, idx]$, 可能的操作有两种:

① 把第 $idx+1$ 个集装箱装上轮船

② 把第 $idx+1$ 个集装箱不装轮船

得到新的状态 $X' = [wt', idx + 1]$

4. 问题解和解空间

满足约束条件的任何 n 维0-1向量 $[x_1, x_2, \dots, x_n]$ 都是一个可行解, 解空间可以用一棵二叉树来表示

广度优先搜索解法

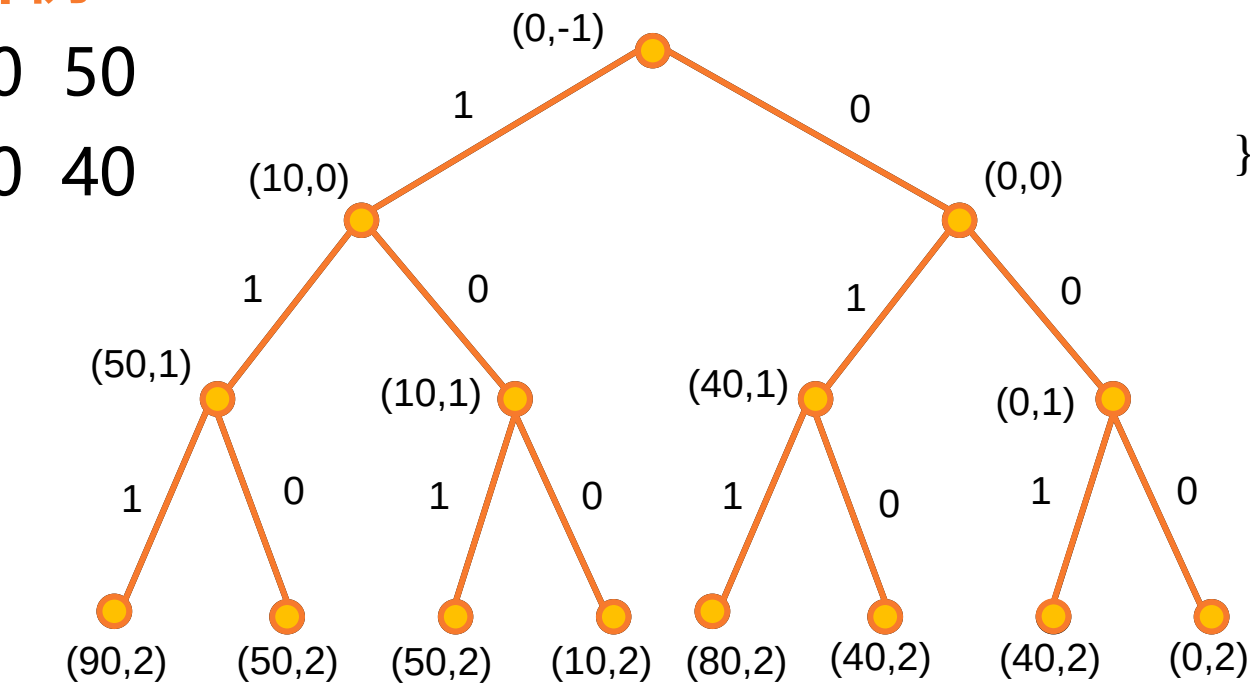
- 辅助数据结构队列
- 结点的Struct
- 扩展子结点方法

```
using namespace std;
struct node {
    int Wt;      //装载的重量
    int idxBox; //当前被处理的集装箱编号
};             //队列中的结点定义
int globalWeight[MaxBox],globalNum,globalC1;
int loadingBFS() {
    int maxWt = 0; //最优装载量
    queue <node> que; //队列定义
    node headNode,sonNode;
    headNode.Wt=0;   headNode.idxBox = -1; //初始状态结点
    que.push(headNode);
    for(; !que.empty(); que.pop()){
        headNode = que.front(); //取出队列首结点
        if(headNode.idxBox == globalNum) { //得到叶子结点, 或n维-1向量
            if((headNode.Wt <= globalC1)&&(headNode.Wt > maxWt)) {
                maxWt = headNode.Wt; //更优的解
            }
        } else { //扩展所有子结点
            sonNode.idxBox = headNode.idxBox + 1;
            sonNode.Wt = headNode.Wt + globalWeight[headNode.idxBox + 1];
            que.push(sonNode); //左子结点
            sonNode.Wt = headNode.Wt;
            que.push(sonNode); //右子结点
        }
    }
    return maxWt;
}
```

输入样例:

3 50 50
10 40 40

```
struct node {  
    int Wt; //装载的重量  
    int idxBox; //当前被处理的集装箱编号  
}; //队列中的结点定义
```



queue

判断条件

$Wt \leq \text{globalC1} \ \&\& \ Wt > \text{maxWt}$

(0,-1)	(10,0)	(0,0)	(50,1)	(10,1)	(40,1)	(0,1)	(90,2)	(50,2)	(50,2)	(10,2)	(80,2)	(40,2)	(40,2)	(0,2)
--------	--------	-------	--------	--------	--------	-------	--------	--------	--------	--------	--------	--------	--------	-------

↑
que.front

↓
maxWt = 50

分支限界算法

1) **约束函数剪枝** 用于剪除那些**不可能导出可行解**的分支。给定任意状态 $[wt, idx]$, 怎么判断其子结点是否可行?

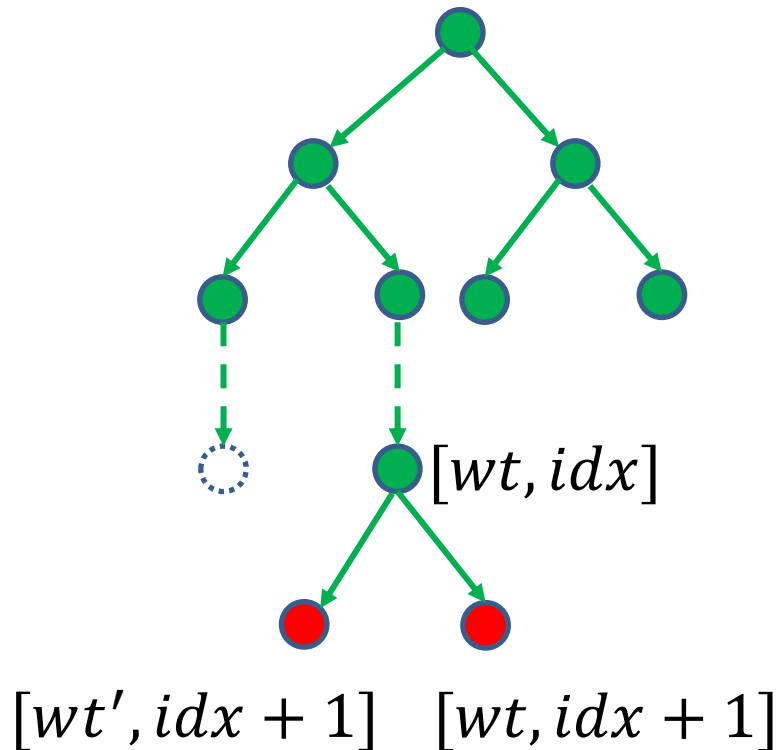
■ 左子结点

$$wt' = wt + w_{idx+1}$$

$$wt' > C_1$$

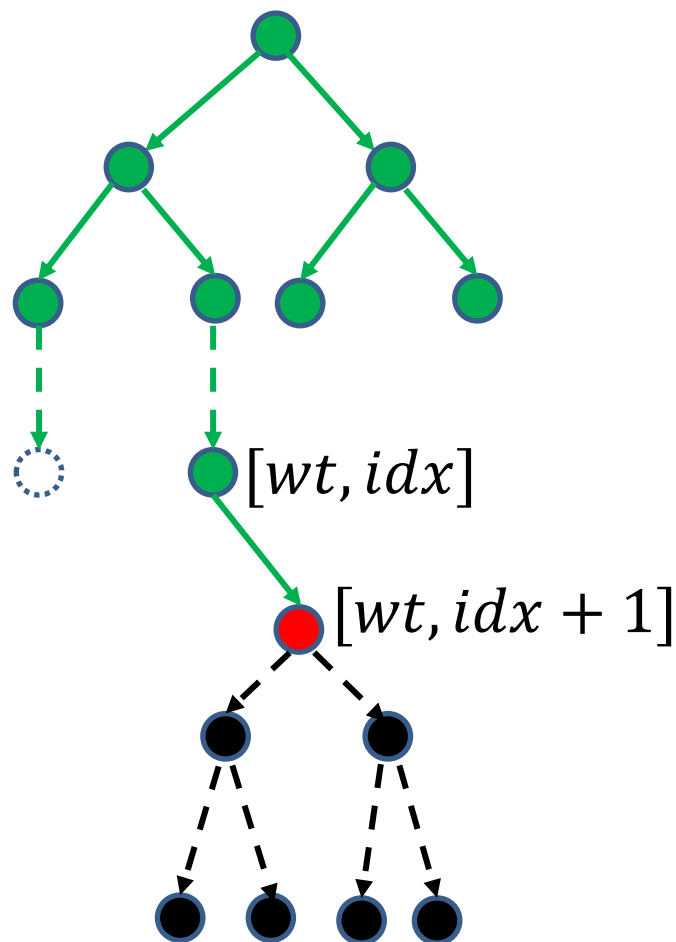
■ 右子结点

$$wt \leq C_1$$



分支限界算法

2) **限界函数剪枝** 用于剪除那些**不可能导出最优解**的分支。给定任意状态 $X = [wt, idx]$ ，怎么判断其子结点是否可能包含最优解？



■ 右子结点

假设 $X_s = [X, X']$ 为包含 X 的解向量，

其中 $X = [x_1, x_2, \dots, x_{idx}, 0]$, $X' = [x_{idx+2}, \dots, x_n]$

$$\begin{aligned} Wt(X_s) &= \sum_{i=1}^{idx+1} w_i x_i + \sum_{i=idx+2}^n w_i x_i \\ &= wt + \mathbf{Wt}(X') \end{aligned}$$

$$\mathbf{Wt}(X) + \mathbf{Bd}(X') \leq \mathbf{MaxWt}$$

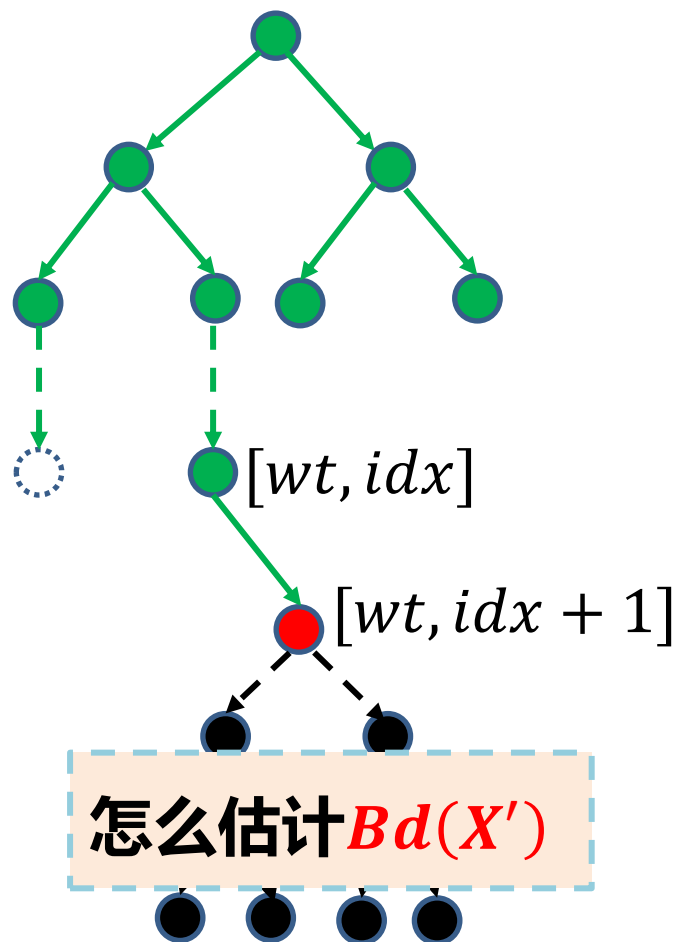
$Wt(X')$ 未知!

➤ 估计 $Wt(X')$ 的**上限** $Bd(X')$ ，即有 $Bd(X') \geq Wt(X')$

➤ 记录当前已得到的最优解，记为 $MaxWt$

分支限界算法

2) **限界函数剪枝** 用于剪除那些**不可能导出最优解**的分支。给定任意状态 $X = [wt, idx]$ ，怎么判断其子结点是否可能包含最优解？



■ 右子结点

$$Wt(X) + Bd(X') \leq MaxWt$$

$$\triangleright Bd(X') = (n - k - 1) * maxW$$

$$\triangleright Bd(X') = \sum_{i=k+2}^n w_i$$

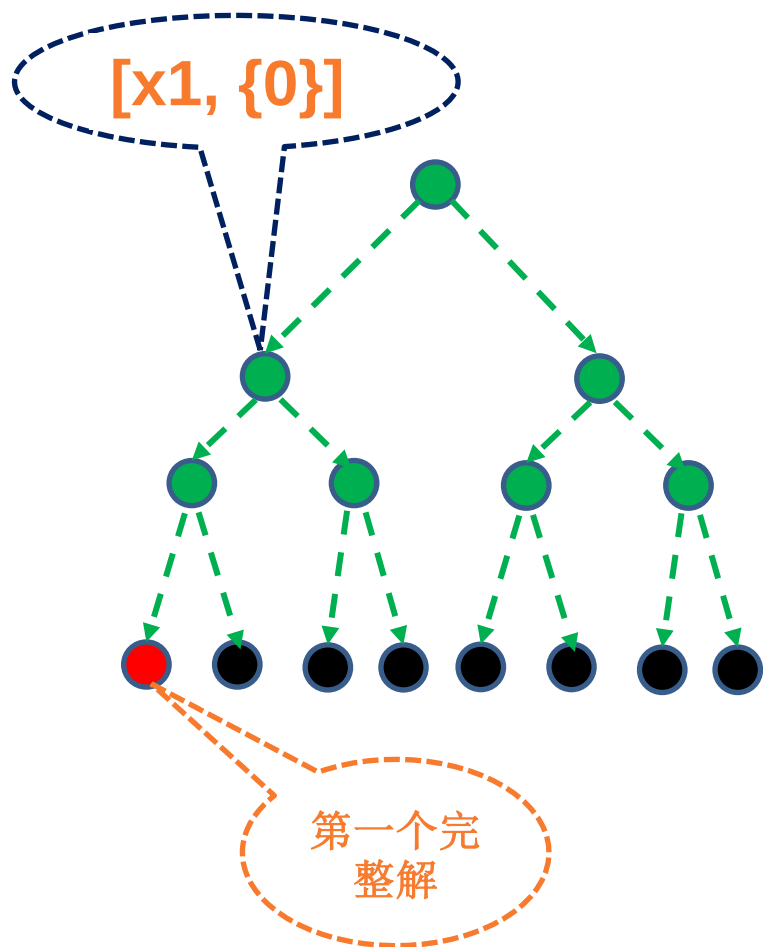
■ 左子结点

$$wt' = wt + w_{idx+1}$$

装载重量的上限等于其父结点的上限，不需判定！

分支限界算法

2) **限界函数剪枝** 用于剪除那些**不可能导出最优解**的分支。给定任意状态 $X = [wt, idx]$ ，怎么判断其子结点是否可能包含最优解？



$$Wt(X) + Bd(X') \leq MaxWt$$

- ✓ $Wt(X)$ 已知
- ✓ $Bd(X')$ 可估计
- ✓ $MaxWt$ 怎么获取？

方案1: 得到第一个**完整解**向量时，类似于回溯算法

方案2: 得到**第一个可行解**时

```

#include <math.h>
#include<stdio.h>
#include<queue>
#define MaxBox 1000
using namespace std;
struct node {
    int Wt;    //装载的重量
    int Bd;    //剩余集装箱总重量
    int idxBox; //当前被处理的集装箱编号
}; //队列中的结点定义
int
globalWeight[MaxBox],globalNum,globalTotalWt,global
C1; //全局变量
int LoadingBranchBound() {
    int maxWt = 0; //最优装载量
    queue <node> que; //队列定义
    node headNode,sonNode;
    headNode.Wt=0;
    headNode.Bd=globalTotalWt;
    headNode.idxBox = -1; //初始状态结点
    que.push(headNode);

```

```

for(; !que.empty(); que.pop()){
    headNode = que.front(); //取队列首结点
    if(headNode.idxBox == globalNum) { //得到叶子结点, 或n维-1向量
        if((headNode.Wt <= globalC1)&&(headNode.Wt > maxWt)) {
            maxWt = headNode.Wt;
        }
    } else { //扩展所有子结点
        sonNode.idxBox = headNode.idxBox + 1;
        sonNode.Bd = headNode.Bd - globalWeight[headNode.idxBox + 1];
        sonNode.Wt = headNode.Wt + globalWeight[headNode.idxBox + 1];
        if(sonNode.Wt <= globalC1){ //约束条件剪枝
            que.push(sonNode); //左子结点
            maxWt = sonNode.Wt > maxWt ? sonNode.Wt:maxWt ; //最优值
        }
        sonNode.Wt = headNode.Wt;
        if(sonNode.Wt + sonNode.Bd > maxWt) //限界条件剪枝
            que.push(sonNode); //右子结点
    }
}
return maxWt;
}

```

剪枝策略:

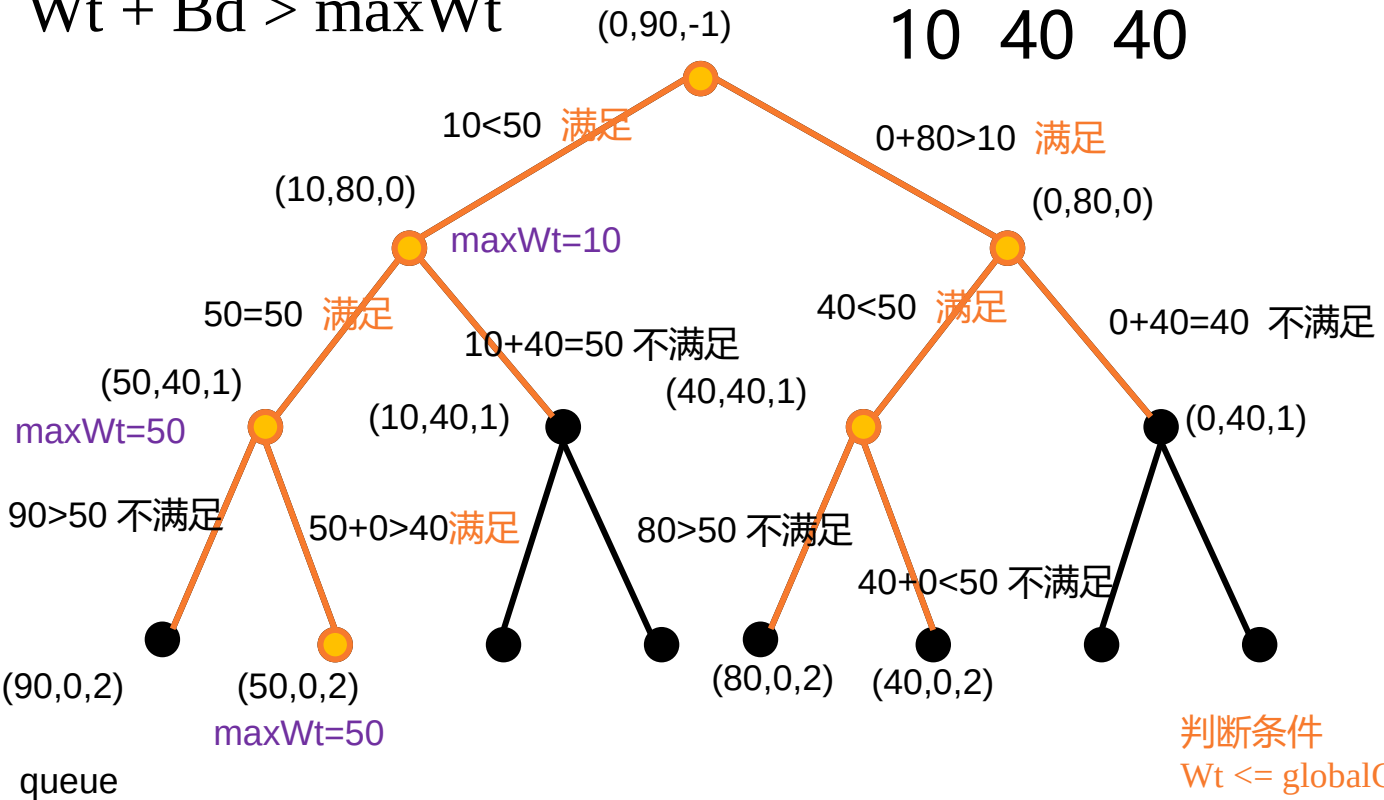
左节点: $Wt \leq globalC1 = 50$

右节点: $Wt + Bd > maxWt$

输入样例:

3 50 50
10 40 40

```
struct node {  
    int Wt; //装载的重量  
    int Bd; //剩余集装箱总重量  
    int idxBox; //当前被处理的集装箱编号  
}; //队列中的结点定义
```



(0,90,-1)	(10,80,0)	(0,80,0)	(50,40,1)	(40,40,1)	(50,0,2)
-----------	-----------	----------	-----------	-----------	----------

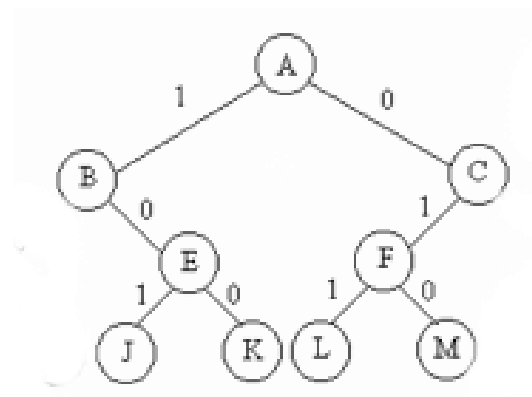
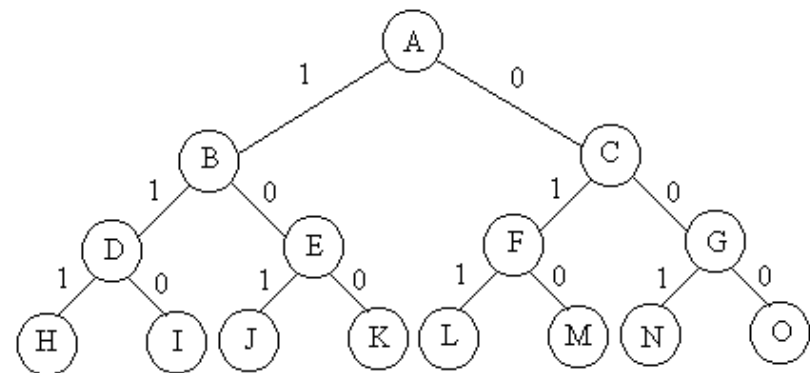
↑
que.front

小结

□ 常用的搜索策略

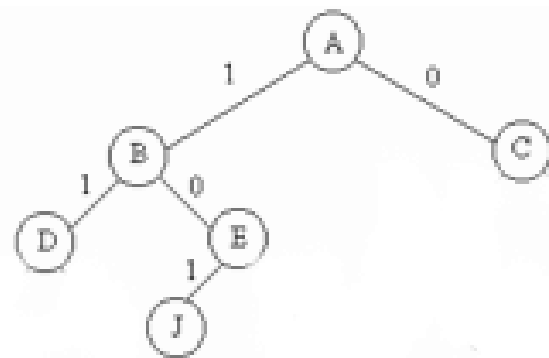
1) 基于枚举的通用搜索

- ✓ 深度优先搜索
- ✓ 广度优先搜索



2) 基于“枚举+优化”的通用搜索

- ✓ 回溯算法
- ✓ 分支限界法



关于回溯法和分支限界法，以下_____是不正确的描述。

- ☒ A 在回溯法中，每个活结点只有一次机会成为扩展结点。
- ☐ B 在分支限界法中，活结点一旦成为扩展结点，将一次性生成其所有的孩子结点，其中导致不可行解或导致非最优解的孩子结点被舍弃，其余孩子结点加入活结点表中。
- ☐ C 回溯法采用深度优先的结点生成策略。
- ☐ D 分支限界法采用广度优先或最小耗费优先（最大效益优先）的结点生成策略

提交

分支限界法中的限界函数用于剪枝，以减少搜索空间的大小。
请问该说法是否正确？

☐ A 错误

☒ B 正确

提交

谢谢 THANKS